

Maths for DSA — Complete Placement & CP Guide

A comprehensive guide covering all essential mathematical algorithms and concepts needed for SDE placements, online assessments, and competitive programming.

1. GCD (Greatest Common Divisor)

1. Overview

GCD of two numbers is the largest positive integer that divides both numbers without leaving a remainder. For example, $\text{GCD}(12, 18) = 6$ because 6 is the largest number that divides both 12 and 18.

2. Intuition

The GCD of two numbers is the biggest number that can perfectly divide both. Think of it as finding the largest common factor.

Analogy: You have 12 apples and 18 oranges. You want to pack them into gift baskets such that each basket has the same number of apples and the same number of oranges, with nothing left over. The GCD (6) tells you the maximum number of baskets you can make — each basket gets 2 apples and 3 oranges.

3. When to Use It

- Finding the largest number dividing two or more numbers
- Simplifying fractions (numerator/GCD, denominator/GCD)
- Problems involving divisibility and common factors
- As a building block for LCM, modular arithmetic, and number theory
- Checking if two numbers are coprime ($\text{GCD} == 1$)

4. When Not to Use It

- When you need all common divisors (use divisor enumeration instead)
- When the built-in is available (no need to reimplement)
- For floating-point numbers — GCD is defined for integers

5. Core Concepts

Euclidean Algorithm

The most efficient method to compute GCD. Based on $\text{gcd}(a, b) = \text{gcd}(b, a \% b)$.

Example: $\text{gcd}(48, 18)$:

- $48 \% 18 = 12 \rightarrow \text{gcd}(18, 12)$
- $18 \% 12 = 6 \rightarrow \text{gcd}(12, 6)$
- $12 \% 6 = 0 \rightarrow \text{gcd}(6, 0) = 6$

$$\text{gcd}(a, 0) = a$$

Base case: any number's GCD with 0 is the number itself.

6. Step-by-Step Algorithm

1. If $b == 0$, return a .
2. Compute $r = a \% b$.
3. Set $a = b, b = r$.
4. Repeat from step 1.

7. Dry Run

Step	a	b	a % b
1	48	18	12
2	18	12	6
3	12	6	0
4	6	0	—

Output: 6

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

int gcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

int gcdRec(int a, int b) {
    if (b == 0) return a;
    return gcdRec(b, a % b);
}

int gcdMultiple(vector<int>& arr) {
    int result = arr[0];
    for (int i = 1; i < arr.size(); i++) {
        result = gcd(result, arr[i]);
        if (result == 1) return 1;
    }
    return result;
}
```

```
int main() {
    cout << gcd(48, 18) << "\n"; // 6
    cout << gcdMultiple({12, 18, 24}) << "\n"; // 6
    return 0;
}
```

9. Python Implementation

```
def gcd(a: int, b: int) -> int:
    while b:
        a, b = b, a % b
    return a

def gcd_rec(a: int, b: int) -> int:
    return a if b == 0 else gcd_rec(b, a % b)

from math import gcd as math_gcd
```

10. Code Explanation

- **Base case** ($b == 0$): GCD of any number with 0 is the number itself.
- **Remainder step**: $a \% b$ gives remainder. The property $\text{gcd}(a, b) = \text{gcd}(b, a \% b)$ holds because any divisor of a and b also divides the remainder.
- **Swap**: Replace (a, b) with $(b, a \% b)$ and continue.

11. Complexity Analysis

Operation	Time Complexity	Space Complexity
Single GCD	$O(\log \min(a, b))$	$O(1)$
GCD of n numbers	$O(n + \log \max_val)$	$O(1)$

12. Common Patterns

Pattern	How to Identify	Approach
GCD of range	"GCD of all subarrays"	Use segment tree or sparse table
Coprime pairs	"pairwise coprime", "GCD = 1"	Iterate through factors, use sieve
Fraction simplification	"simplify fraction"	Divide num/den by their GCD
Cyclic rotations	"rotate array by k "	GCD of n and k gives number of cycles

13. Common Mistakes

- **Ignoring negative numbers**: GCD is always non-negative. Use `abs()`.

- **Not handling zero:** $\text{gcd}(a, 0)$ should return $|a|$, not 0 .
- **Confusing with LCM**
- **Using subtraction instead of modulo:** Much slower for large numbers with big disparity.

14. Edge Cases

Input	Expected	Reason
$\text{gcd}(0, 0)$	0	Convention
$\text{gcd}(0, 5)$	5	$\text{gcd}(0, a) = a$
$\text{gcd}(1, \text{any})$	1	1 divides everything
$\text{gcd}(\text{neg}, \text{pos})$	positive	Use absolute values

15. Variations

Binary GCD (Stein's Algorithm)

Uses bit operations instead of modulo. Faster on some hardware.

Extended Euclidean Algorithm

Finds integers x, y such that $ax + by = \text{gcd}(a, b)$. Essential for modular inverse.

16. Related Algorithms

- **LCM** = $a * b / \text{gcd}(a, b)$
- **Extended Euclidean** — generalization
- **Euler Totient** — uses GCD indirectly

17. Practice Problems

Easy

Problem	Platform	Main Idea
Find GCD of two numbers	LeetCode 1979	Direct GCD
GCD of Array	LeetCode 914	Multiple numbers

Medium

Problem	Platform	Main Idea
Fraction Addition and Subtraction	LeetCode 592	GCD for simplification
GCD Sort of an Array	LeetCode 1998	GCD + DSU

Hard

Problem	Platform	Main Idea
Count Subarrays with GCD = 1	Codeforces	Map merging technique

18. Interview Explanation

"GCD is the largest number that divides two numbers without remainder. I use the Euclidean algorithm — repeatedly replace the larger number with the remainder of dividing it by the smaller, until we reach zero. This works because any common divisor of a and b also divides $a \% b$. The algorithm runs in $O(\log \min(a, b))$, making it very efficient."

19. Revision Notes

- **Key formula:** $\text{gcd}(a, b) = \text{gcd}(b, a \% b)$ until $b = 0$
- **Built-in:** `__gcd()`, `std::gcd`, `math.gcd`
- **Complexity:** $O(\log \min(a, b))$
- **Edge cases:** $\text{gcd}(0, a) = a$, $\text{gcd}(1, a) = 1$

20. Final Cheat Sheet

Aspect	Details
When to use	Divisibility, fractions, common factors
Algorithm	Euclidean: <code>while(b) { b = a % b }</code>
Complexity	$O(\log \min(a, b))$
Key code	<code>while(b) { a %= b; swap(a,b); }</code>
Formula	$\text{gcd}(a,b) \times \text{lcm}(a,b) = a \times b$

2. LCM (Least Common Multiple)

1. Overview

LCM of two numbers is the smallest positive integer divisible by both numbers. $\text{LCM}(4, 6) = 12$.

2. Intuition

Analogy: Two buses start from the same stop. Bus A comes every 4 minutes, Bus B every 6 minutes. When will they arrive together again? After $\text{LCM}(4, 6) = 12$ minutes.

Key insight: If $g = \text{gcd}(a, b)$, then $a = g \times x$ and $b = g \times y$ where x and y are coprime. $\text{LCM} = g \times x \times y = (a \times b) / g$.

3. When to Use It

- Finding when events align
- Adding fractions (LCM of denominators)

- "Smallest number divisible by..."
- Cyclic patterns and scheduling

4. When Not to Use It

- When GCD is sufficient (many "divisible by both" problems need GCD checks)
- When numbers are very large and $a * b$ overflows

5. Core Concepts

Relation with GCD

$$\text{LCM}(a, b) \times \text{GCD}(a, b) = a \times b$$

LCM of Coprime Numbers

If $\text{gcd}(a, b) = 1$, then $\text{LCM}(a, b) = a \times b$.

6. Step-by-Step Algorithm

1. Compute $g = \text{gcd}(a, b)$.
2. Return $(a / g) * b$. (Divide first to avoid overflow.)

7. Dry Run

Step	Operation	Result
1	Input: a=12, b=18	—
2	$g = \text{gcd}(12, 18)$	6
3	$(12 / 6) * 18$	$2 * 18 = 36$

Output: 36

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

int gcd(int a, int b) {
    while (b) { a %= b; swap(a, b); }
    return a;
}

long long lcm(int a, int b) {
    return (a / gcd(a, b)) * 1LL * b;
}

long long lcmMultiple(vector<int>& arr) {
    long long result = 1;
    for (int num : arr) result = lcm(result, num);
}
```

```

        return result;
    }

    int main() {
        cout << lcm(12, 18) << "\n";    // 36
        cout << lcm(7, 13) << "\n";    // 91
        return 0;
    }

```

9. Python Implementation

```

from math import gcd

def lcm(a: int, b: int) -> int:
    return a // gcd(a, b) * b

def lcm_multiple(arr: list) -> int:
    result = 1
    for num in arr:
        result = lcm(result, num)
    return result

```

10. Code Explanation

- **Division first:** $a / \text{gcd} * b$ prevents overflow. If we wrote $a * b / \text{gcd}$, the multiplication could overflow even if the final result fits.
- **Cast to long long:** Use `1LL *` for safe multiplication.

11. Complexity Analysis

Operation	Time Complexity	Space Complexity
Single LCM	$O(\log \min(a, b))$	$O(1)$
LCM of n numbers	$O(n \log \max_val)$	$O(1)$

12. Common Patterns

Pattern	How to Identify	Approach
Fraction sum	"Add fractions"	LCM of denominators
Periodic alignment	"meet again", "both happen on"	LCM of periods
Smallest divisible number	"smallest number divisible by all"	LCM of the array

13. Common Mistakes

- **Overflow:** $a * b$ before division causes overflow. Always divide first.

- **Not using long long:** LCM of moderate numbers exceeds 32-bit int.
- **Zero input:** LCM with 0 is undefined. Handle separately.

14. Edge Cases

Input	Expected	Reason
$\text{lcm}(1, \text{any})$	any	1 divides everything
$\text{lcm}(7, 13)$	91	Coprime
$\text{lcm}(100, 100)$	100	Equal numbers

15. Variations

LCM of Fractions

$\text{LCM}(\text{num1}, \text{num2}) / \text{GCD}(\text{den1}, \text{den2})$. Rare in CP.

16. Related Algorithms

- **GCD** — LCM is computed via GCD
- **Prime factorization** — LCM exponent = max exponent of each prime

17. Practice Problems

Easy

Problem	Platform	Main Idea
LCM of Two Numbers	GFG	Direct formula
Smallest Multiple	Project Euler 5	LCM of 1..20

Medium

Problem	Platform	Main Idea
Fraction Addition	LeetCode 592	LCM for denominators

18. Interview Explanation

"LCM is the smallest number divisible by both given numbers. I use $\text{LCM}(a, b) = a / \text{GCD}(a, b) * b$, computing GCD first to prevent overflow. This takes $O(\log \min(a, b))$ time. For multiple numbers, I compute it pairwise."

19. Revision Notes

- **Formula:** $\text{LCM}(a, b) = a / \text{gcd}(a, b) * b$
- **Key property:** $\text{gcd}(a, b) \times \text{lcm}(a, b) = a \times b$
- **Overflow:** Always divide before multiplying

- **Edge:** $\text{lcm}(0, a) = 0$

20. Final Cheat Sheet

Aspect	Details
When to use	"smallest number divisible by both"
Formula	$a / \text{gcd}(a, b) * b$
Complexity	$O(\log \min(a, b))$
Pitfall	Overflow in $a * b$

3. Euclidean Algorithm

1. Overview

The Euclidean algorithm efficiently computes GCD of two numbers. It repeatedly uses $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$.

2. Intuition

Analogy: You have a 48m and 18m rope. You want the longest tape that can measure both exactly.

- Cut 48m using 18m: $48 = 2 \times 18 + 12$ (remainder 12m)
- Cut 18m using 12m: $18 = 1 \times 12 + 6$ (remainder 6m)
- Cut 12m using 6m: $12 = 2 \times 6 + 0$ (no remainder)
- Last non-zero remainder (6m) is your answer.

3. When to Use It

- Computing GCD of any two integers
- As a subroutine in modular arithmetic (modular inverse)
- In any number theory problem

4. When Not to Use It

- When built-in `__gcd()` is available — same algorithm, no need to reimplement

5. Core Concepts

Modulo Operation

$a \bmod b$ gives remainder. This is the key shrinking operation.

Invariant

Throughout the algorithm, $\text{gcd}(a, b)$ remains invariant.

6. Step-by-Step Algorithm

1. While $b \neq 0$: $a, r = a \% b$ $b, a = b, r$
2. Return a

7. Dry Run

gcd(105, 252):

Iteration	a	b	r = a % b
1	252	105	42
2	105	42	21
3	42	21	0
4	21	0	—

Output: 21

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

int gcd(int a, int b) {
    while (b) {
        a %= b;
        swap(a, b);
    }
    return a;
}

int main() {
    cout << gcd(252, 105) << "\n"; // 21
    cout << gcd(0, 10) << "\n";    // 10
    return 0;
}
```

9. Python Implementation

```
def gcd(a: int, b: int) -> int:
    while b:
        a, b = b, a % b
    return a
```

10. Code Explanation

- **Loop condition:** `while (b)` continues until `b` becomes 0.
- **swap(a, b):** Cleaner than a temp variable after computing `a %= b`.

11. Complexity Analysis

Aspect	Value
Time	$O(\log \min(a, b))$
Space (iter)	$O(1)$
Space (rec)	$O(\log \min(a, b))$ for call stack

12. Common Patterns

Pattern	How to Identify	Approach
Basic GCD	"gcd of two numbers"	Direct Euclidean
GCD in fraction reduction	"simplify a/b"	Divide by gcd(num, den)

13. Common Mistakes

- **No swap:** Forgetting to swap causes infinite loop.
- **Using modulo on zero:** `a % 0` causes division by zero.
- **Negative mod behavior:** Take absolute values.

14. Edge Cases

Input	Expected	Reason
gcd(0, 0)	0	Convention
gcd(0, 10)	10	$\text{gcd}(0, a) = a$
gcd(-12, -18)	6	GCD is positive

15. Variations

Extended Euclidean Algorithm

Finds x, y such that $ax + by = \text{gcd}(a, b)$. Used for modular inverse.

Binary GCD (Stein's Algorithm)

Uses bit operations instead of modulo.

16. Related Algorithms

- **Extended Euclidean Algorithm** — the most important extension
- **Stein's Algorithm** — alternative implementation

17. Practice Problems

Easy

Problem	Platform	Difficulty
Find GCD	GFG	Easy
GCD and LCM	CSES 1071	Easy

Medium

Problem	Platform	Difficulty
Euclid's Game	Codeforces	Medium

18. Interview Explanation

"The Euclidean algorithm computes GCD by repeatedly applying $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$. It works because any divisor of both a and b must also divide their remainder. The algorithm runs in $O(\log \min(a, b))$ time."

19. Revision Notes

- **Core idea:** $\text{gcd}(a, b) = \text{gcd}(b, a \% b)$, repeat until $b = 0$
- **Complexity:** $O(\log \min(a, b))$
- **Edge:** $\text{gcd}(0, a) = a$
- **Extension:** Extended Euclidean $\rightarrow$ modular inverse

20. Final Cheat Sheet

Aspect	Details
Algorithm	<code>while(b) { a %= b; swap(a,b); }</code>
Complexity	$O(\log \min(a,b))$
Key insight	$\text{gcd}(a,b) = \text{gcd}(b, a \bmod b)$
Extension	Extended Euclidean $\rightarrow$ modular inverse

4. Prime Checking

1. Overview

Determine whether a given integer $n > 1$ is prime — divisible only by 1 and itself.

2. Intuition

Why $\text{sqrt}(n)$ works: If $n = a \times b$ and both a and $b > \text{sqrt}(n)$, then $a \times b > n$. So at least one factor $\leq \text{sqrt}(n)$. We only need to check up to $\text{sqrt}(n)$.

3. When to Use It

- Determining if a single number is prime
- As a subroutine in prime factorization

4. When Not to Use It

- For many numbers (use sieve instead — $O(1)$ per query)
- For very large numbers ($> 10^{12}$) — use Miller-Rabin

5. Core Concepts

Trial Division

Checking divisibility by numbers up to $\text{sqrt}(n)$.

Optimizations:

- Check 2 separately, then only odd numbers
- Check 2 and 3, then numbers of form $6k \pm 1$

6. Step-by-Step Algorithm

1. If $n \leq 1$, return false.
2. If $n \leq 3$, return true.
3. If $n \% 2 == 0$ or $n \% 3 == 0$, return false.
4. For $i = 5$ to $\text{sqrt}(n)$ step 6:
 - If $n \% i == 0$ or $n \% (i + 2) == 0$, return false.
5. Return true.

7. Dry Run

Check if 97 is prime:

$\text{sqrt}(97) \approx 9.8$. Check $i=5$ ($97\%5=2$), $i+2=7$ ($97\%7=6$). $i=11 > 9$, stop. 97 is prime.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

bool isPrime(int n) {
    if (n <= 1) return false;
    if (n <= 3) return true;
    if (n % 2 == 0 || n % 3 == 0) return false;
    for (int i = 5; i * i <= n; i += 6) {
        if (n % i == 0 || n % (i + 2) == 0) return false;
    }
    return true;
}
```

```

    }
    return true;
}

int main() {
    cout << isPrime(97) << "\n"; // 1
    cout << isPrime(100) << "\n"; // 0
    return 0;
}

```

9. Python Implementation

```

def is_prime(n: int) -> bool:
    if n <= 1: return False
    if n <= 3: return True
    if n % 2 == 0 or n % 3 == 0: return False
    i = 5
    while i * i <= n:
        if n % i == 0 or n % (i + 2) == 0: return False
        i += 6
    return True

```

10. Code Explanation

- **$6k \pm 1$ optimization:** All primes beyond 3 are of form $6k \pm 1$. Cuts iterations by 2/3.
- **$i * i \leq n$:** Avoids floating-point sqrt. Watch for overflow in i^2 .

11. Complexity Analysis

Variant	Time Complexity	Space
Basic trial division	$O(\sqrt{n})$	$O(1)$
$6k \pm 1$ optimization	$O(\sqrt{n}/3)$	$O(1)$

12. Common Patterns

Pattern	How to Identify	Approach
Single primality	"is 97 prime?"	$O(\sqrt{n})$ trial division
Find next prime	"smallest prime $> n$ "	Increment + check

13. Common Mistakes

- **Forgetting $n \leq 1$:** 1 is not prime
- **$i * i$ overflow:** Use $i \leq n / i$ instead
- **Missing 2 as prime:** 2 is prime

14. Edge Cases

Input	Result	Reason
1	false	Not prime
2	true	Prime
3	true	Prime
97	true	Prime

15. Variations

Miller-Rabin Primality Test

Probabilistic test for large numbers. $O(k \log^3 n)$.

16. Related Algorithms

- **Sieve of Eratosthenes** — all primes up to n
- **Miller-Rabin** — for larger numbers

17. Practice Problems

Easy

Problem	Platform	Difficulty
Prime Number	GFG	Easy
Count Primes	LeetCode 204	Easy-Medium

18. Interview Explanation

"For checking if a number is prime, I use trial division up to $\sqrt{n}$. The optimization is checking 2 separately, then only odd numbers. A further optimization is $6k \pm 1$ since all primes beyond 3 are of that form. $O(\sqrt{n})$ time. For many queries, I'd use a sieve."

19. Revision Notes

- $n \leq 1 \rightarrow$ not prime. $n = 2$ or $3 \rightarrow$ prime.
- $O(\sqrt{n})$ time using $6k \pm 1$ optimization.
- Sieve if multiple queries needed.

20. Final Cheat Sheet

Aspect	Details
Algorithm	Up to $\sqrt{n}$, skip evens, $6k \pm 1$
Complexity	$O(\sqrt{n})$

Aspect	Details
Key code	<code>`for(i=5; i*i<=n; i+=6) if(n%i==0`</code>
For many queries	Use Sieve of Eratosthenes

5. Sieve of Eratosthenes

1. Overview

Efficiently find all prime numbers up to a given limit n by systematically marking multiples of each prime starting from 2.

2. Intuition

Analogy: Lockers numbered 1 to n , all closed (prime = closed). Starting from 2, open every 2nd locker. Then take next closed (3), open every 3rd. Continue. At the end, closed lockers are primes.

Why it works: When we reach number p , if unmarked, no smaller number divides it $\rightarrow p$ is prime.

3. When to Use It

- Finding all primes up to n (10^7 or 10^8 with memory optimization)
- Answering multiple "is prime?" queries in $O(1)$
- Precomputing primes for factorization
- Prime counts, prime sums

4. When Not to Use It

- For checking a single number (trial division is better)
- When n is huge (10^9+) — memory prohibitive. Use segmented sieve.

5. Core Concepts

Marking Multiples

For each prime p , mark multiples starting from p^2 (smaller ones already marked by smaller primes).

Time Complexity

$O(n \log \log n)$.

6. Step-by-Step Algorithm

1. Create boolean array `prime[0..n]`, all true.
2. Set `prime[0] = prime[1] = false`.
3. For `p = 2; p * p <= n; p++`:
 - If `prime[p]`:
 - For `i = p * p; i <= n; i += p`: set `prime[i] = false`.

4. All i where `prime[i]` true are primes.

7. Dry Run

Sieve up to 20:

p	Marked
2	4, 6, 8, 10, 12, 14, 16, 18, 20
3	9, 12, 15, 18
5	25 > 20, skip

Primes: [2, 3, 5, 7, 11, 13, 17, 19]

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

vector<bool> sieve(int n) {
    vector<bool> isPrime(n + 1, true);
    isPrime[0] = isPrime[1] = false;
    for (int p = 2; p * p <= n; p++) {
        if (isPrime[p]) {
            for (int i = p * p; i <= n; i += p) {
                isPrime[i] = false;
            }
        }
    }
    return isPrime;
}

vector<int> sievePrimes(int n) {
    vector<bool> isPrime(n + 1, true);
    vector<int> primes;
    isPrime[0] = isPrime[1] = false;
    for (int p = 2; p <= n; p++) {
        if (isPrime[p]) {
            primes.push_back(p);
            if ((long long)p * p <= n) {
                for (int i = p * p; i <= n; i += p) {
                    isPrime[i] = false;
                }
            }
        }
    }
    return primes;
}
```

9. Python Implementation

```
def sieve(n: int):
    is_prime = [True] * (n + 1)
    is_prime[0] = is_prime[1] = False
    for p in range(2, int(n**0.5) + 1):
        if is_prime[p]:
            for i in range(p * p, n + 1, p):
                is_prime[i] = False
    return is_prime
```

10. Code Explanation

- **Start from p^2 :** All smaller multiples marked by earlier primes.
- **$p^2 \leq n$:** Only mark if $p^2 \leq n$.

11. Complexity Analysis

Operation	Time Complexity	Space Complexity
Standard Sieve	$O(n \log \log n)$	$O(n)$

12. Common Patterns

Pattern	How to Identify	Approach
Count primes $\leq n$	"number of primes"	Sieve + count true values
Generate primes for factorization	"prime factors of all numbers up to n"	Sieve + SPF

13. Common Mistakes

- **Starting loop from p :** Correct but slower. Start from p^2 .
- **Using `vector<int>` for sieve:** Too memory-heavy. Use `vector<bool>` or bitset.
- **Not using long long:** $p * p$ can overflow int for $p > 46340$.

14. Edge Cases

Input n	Behavior
0 or 1	Empty primes list
2	Returns {2}

15. Variations

Segmented Sieve

Divide range $[L, R]$ into blocks. For each block, use primes up to $\sqrt{R}$ to mark composites.

Linear Sieve (Sieve of Euler)

Each composite marked exactly once by its SPF. $O(n)$ time.

16. Related Algorithms

- **Prime factorization** — uses sieve for SPF
- **Miller-Rabin** — for larger numbers

17. Practice Problems

Easy

Problem	Platform	Main Idea
Count Primes	LeetCode 204	Direct sieve usage

Medium

Problem	Platform	Main Idea
Prime Sum	GFG	Sieve + two-pointer

Hard

Problem	Platform	Main Idea
Segmented Sieve	SPOJ PRIME1	Range sieve

18. Interview Explanation

"The Sieve of Eratosthenes finds all primes up to n in $O(n \log \log n)$ time. I create a boolean array, initially all true. For each prime p starting from 2, I mark multiples starting from p^2 as false. The key optimization is starting from p^2 — smaller multiples are already handled. Memory is $O(n)$, but I can use bitset or segmented sieve for larger limits."

19. Revision Notes

- **Initialize:** `isPrime[0..n] = true`, set `0, 1 = false`
- **Loop:** `for(p=2; p*p<=n; p++)` mark from `p*p` to `n` step `p`
- **Complexity:** $O(n \log \log n)$
- **Linear sieve:** Each composite marked once by its SPF

20. Final Cheat Sheet

Aspect	Details
When to use	All primes up to n , many queries
Algorithm	Mark multiples of each prime from p^2

Aspect	Details
Complexity	$O(n \log \log n)$ time, $O(n)$ memory
Key code	<code>for(i=p*p; i<=n; i+=p) isPrime[i]=false</code>
For large n	Segmented sieve

6. Modular Arithmetic

1. Overview

A system of arithmetic for integers where numbers "wrap around" upon reaching a certain value — the modulus. Two numbers are congruent modulo M if they leave the same remainder when divided by M .

2. Intuition

Analogy: A clock. If it's 10 AM and you add 5 hours, you get 3 PM (not 15). This is modulo 12: $(10 + 5) \bmod 12 = 3$.

Why it matters in programming: Numbers quickly grow too large for 64-bit integers. By taking modulo at each step, we keep numbers manageable.

3. When to Use It

- When problem says "return answer modulo 10^9+7 "
- When numbers grow too large during computation
- In cryptography
- In competitive programming (almost every combinatorial problem)

4. When Not to Use It

- When you need actual values for comparison (modulo destroys ordering)
- When M is not prime and you need division (need modular inverse)

5. Core Concepts

Properties

- $(a + b) \bmod M = ((a \bmod M) + (b \bmod M)) \bmod M$
- $(a - b) \bmod M = ((a \bmod M) - (b \bmod M) + M) \bmod M$
- $(a \times b) \bmod M = ((a \bmod M) \times (b \bmod M)) \bmod M$
- $(a / b) \bmod M = a \times b^{-1} \bmod M$

Negative Numbers

In C++, $a \% M$ is negative if a is negative. Fix: $((a \% M) + M) \% M$.

Common Moduli

- $10^9 + 7$ (prime) — most common in CP
- 998244353 (prime) — Codeforces

6. Step-by-Step Operations

- Add: $(a \% M + b \% M) \% M$
- Subtract: $(a \% M - b \% M + M) \% M$
- Multiply: $((a \% M) * (b \% M)) \% M$
- Divide: $a \% M * \text{modInverse}(b, M) \% M$

7. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const int MOD = 1e9 + 7;

ll add(ll a, ll b, ll mod = MOD) {
    return (a % mod + b % mod) % mod;
}

ll sub(ll a, ll b, ll mod = MOD) {
    return (a % mod - b % mod + mod) % mod;
}

ll mul(ll a, ll b, ll mod = MOD) {
    return ((a % mod) * (b % mod)) % mod;
}

// Safe multiplication for mod up to 10^18
ll mulMod(ll a, ll b, ll mod) {
    ll res = 0;
    a %= mod;
    while (b) {
        if (b & 1) res = (res + a) % mod;
        a = (a * 2) % mod;
        b >>= 1;
    }
    return res;
}
```

8. Python Implementation

```
MOD = 10**9 + 7

def add(a: int, b: int, mod: int = MOD) -> int:
    return (a % mod + b % mod) % mod
```

```
def sub(a: int, b: int, mod: int = MOD) -> int:
    return (a % mod - b % mod + mod) % mod

def mul(a: int, b: int, mod: int = MOD) -> int:
    return (a % mod) * (b % mod) % mod
```

9. Complexity Analysis

Operation	Time
mod add/sub	$O(1)$
mod mul	$O(1)$
mod div (inv)	$O(\log \text{MOD})$

10. Common Mistakes

- **Negative mod in C++:** $(-5) \% 3 = -2$. Always add MOD.
- **Not using long long:** Multiplying two ints mod $1e9+7$ can overflow 32-bit.
- **Division without inverse:** $(a \% M) / (b \% M)$ is NOT correct.

11. Edge Cases

Input	Issue	Fix
Negative a	$a \% \text{MOD}$ can be negative	$(a \% \text{MOD} + \text{MOD}) \% \text{MOD}$
$\text{MOD} = 0$	Division by zero	Not valid modulus
MOD not prime	Can't use Fermat for inv	Use extended Euclidean

12. Related Algorithms

- **Modular exponentiation** — compute $a^b \bmod M$
- **Modular inverse** — needed for division
- **Extended Euclidean** — modular inverse without prime MOD

13. Practice Problems

Easy

Problem	Platform	Main Idea
Modulo Arithmetic	GFG	Basic operations

Medium

Problem	Platform	Main Idea
Super Pow	LeetCode 372	Large exponent modulo

14. Interview Explanation

"Modular arithmetic works with remainders. For $+$, $-$, $\times$, we apply modulo at each step. For division, we need modular inverse — with prime modulus M , Fermat's little theorem gives $a^{-1} \equiv a^{(M-2)} \pmod{M}$. In C++, I'm careful with negative numbers."

15. Revision Notes

- $(a + b) \% M = ((a \% M) + (b \% M)) \% M$
- $(a - b) \% M = ((a \% M) - (b \% M) + M) \% M$
- $(a \times b) \% M = ((a \% M) \times (b \% M)) \% M$
- Fix negative: $(x \% M + M) \% M$
- Common MOD: $1e9+7$ (prime)

16. Final Cheat Sheet

Aspect	Details
When to use	Large numbers, DP, combinatorial
Core ops	$+$, $-$, $\times \pmod{M}$; $/$ needs inv
Key code	$((a \% M) + M) \% M$ for negative fix
Inverse	Fermat (prime MOD) or Extended Euclidean

7. Modular Exponentiation

1. Overview

Compute $(a^b) \% M$ in $O(\log b)$ using binary exponentiation (exponentiation by squaring).

2. Intuition

Analogy: To compute a^{25} , note $25 = 11001$ in binary $= 16 + 8 + 1$. So $a^{25} = a^{16} \times a^8 \times a^1$. Compute a^1 , a^2 , a^4 , a^8 , a^{16} by repeated squaring, then multiply needed powers.

Key insight: If b is even: $a^b = (a^{(b/2)})^2$. If b is odd: $a^b = a \times (a^{((b-1)/2)})^2$.

3. When to Use It

- Computing large powers with modulo
- Modular inverse via Fermat ($a^{(M-2)} \pmod{M}$)
- In cryptography
- In combinatorial formulas

4. When Not to Use It

- Without modulo (use big integers via Python)
- For small b where naive loop is fine

5. Step-by-Step Algorithm

1. Initialize `result = 1`, `base = a % MOD`.
2. While `b > 0`:
 - If `b & 1`: `result = (result × base) % MOD`
 - `base = (base × base) % MOD`
 - `b >>= 1`
3. Return `result`.

6. Dry Run

Compute $3^{13} \bmod 7$:

13 in binary: 1101 (bits LSB first: 1, 0, 1, 1).

Iteration	b (binary)	bit	result	base
Start	1101	—	1	3
1	110	1	3	2
2	11	0	3	4
3	1	1	5	2
4	0	1	3	—

Result: 3

7. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const int MOD = 1e9 + 7;

ll modPow(ll a, ll b, ll mod = MOD) {
    ll res = 1;
    a %= mod;
    while (b > 0) {
        if (b & 1) res = (res * a) % mod;
        a = (a * a) % mod;
        b >>= 1;
    }
    return res;
}

int main() {
    cout << modPow(3, 13, 7) << "\n";    // 3
    cout << modPow(2, 1000000) << "\n";  // fast
}
```

```
    return 0;
}
```

8. Python Implementation

```
def mod_pow(a: int, b: int, mod: int = 10**9 + 7) -> int:
    res = 1
    a %= mod
    while b:
        if b & 1:
            res = (res * a) % mod
        a = (a * a) % mod
        b >>= 1
    return res
```

9. Complexity Analysis

Variant	Time Complexity	Space Complexity
Binary Exponentiation	$O(\log b)$	$O(1)$
Recursive	$O(\log b)$	$O(\log b)$ (stack)

10. Common Mistakes

- **Not taking $a \% MOD$** at start
- **Forgetting $\% MOD$** after multiplication
- **Using `pow()` from `<cmath>`:** That's for floating-point

11. Edge Cases

Input	Expected	Note
<code>modPow(0, 5, M)</code>	0	Base 0
<code>modPow(5, 0, M)</code>	1	$\text{Any}^0 = 1$

12. Variations

Matrix Exponentiation

Same idea with matrices. Used for linear recurrences (Fibonacci, etc.).

Large Exponent as String

Reduce exponent using Euler's theorem: $a^b \bmod M = a^{(b \bmod \phi(M))} \bmod M$.

13. Practice Problems

Easy

Problem	Platform	Main Idea
Pow(x, n)	LeetCode 50	Binary exponentiation

Medium

Problem	Platform	Main Idea
Super Pow	LeetCode 372	Large exponent as array

14. Interview Explanation

"Modular exponentiation computes $a^b \bmod M$ in $O(\log b)$ using binary exponentiation. I iterate through bits of b — squaring the base at each step and multiplying into result when the current bit is 1. This is much faster than naive $O(b)$ and is essential for computing modular inverses via Fermat."

15. Revision Notes

- **Binary exponentiation:** `while(b) { if(b&1) res *= a; a *= a; b >>= 1; }`
- **Complexity:** $O(\log b)$
- **Fermat inverse:** $a^{(M-2)} \bmod M$

16. Final Cheat Sheet

Aspect	Details
Algorithm	Binary exponentiation
Complexity	$O(\log b)$
Key code	<code>while(b) { if(b&1) res=res*a%M; a=aa%M; b>>=1; }</code>
Extension	Matrix exponentiation for sequences

8. Factorial Modulo

1. Overview

Compute $n! \% M$ efficiently. Precompute in $O(n)$, $O(1)$ per query.

2. Intuition

If M is prime and $n \geq M$, then $n! \% M = 0$ because M is one of the factors. For $n < M$, precompute: $\text{fact}[i] = \text{fact}[i-1] \times i \% M$.

3. When to Use It

- Computing nCr or nPr modulo M

- In combinatorics problems with modulo
- Multiple factorial values needed

4. When Not to Use It

- When n is small (compute directly)
- When n is huge (use Lucas theorem)

5. Core Concepts

Precomputation

`fact[0] = 1, fact[i] = fact[i-1] * i % MOD.`

Inverse Factorial

`invFact[n] = (n!)-1 mod M.` Compute last using Fermat, then fill backwards.

6. Step-by-Step Algorithm

1. `fact[0] = 1`
2. For $i = 1$ to n : `fact[i] = fact[i-1] * i % MOD`
3. `invFact[n] = modPow(fact[n], MOD-2, MOD)`
4. For $i = n$ down to 1 : `invFact[i-1] = invFact[i] * i % MOD`

7. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const int MOD = 1e9 + 7;
const int MAXN = 1000000;

ll fact[MAXN + 1], invFact[MAXN + 1];

ll modPow(ll a, ll b, ll mod = MOD) {
    ll res = 1; a %= mod;
    while (b) { if (b & 1) res = res * a % mod; a = a * a % mod; b >>= 1; }
    return res;
}

void precomputeFactorials(int n = MAXN) {
    fact[0] = 1;
    for (int i = 1; i <= n; i++) fact[i] = fact[i - 1] * i % MOD;
    invFact[n] = modPow(fact[n], MOD - 2);
    for (int i = n; i >= 1; i--) invFact[i - 1] = invFact[i] * i % MOD;
}

ll nCr(ll n, ll k) {
    if (k < 0 || k > n) return 0;
```

```

        return fact[n] * invFact[k] % MOD * invFact[n - k] % MOD;
    }

    ll nPr(ll n, ll k) {
        if (k < 0 || k > n) return 0;
        return fact[n] * invFact[n - k] % MOD;
    }

```

8. Complexity Analysis

Operation	Time	Space
Precompute factorials	$O(n)$	$O(n)$
nCr/nPr query	$O(1)$	—

9. Common Mistakes

- **Not handling $n \geq \text{MOD}$:** Need Lucas theorem
- **Forgetting $\text{invFact}[0]$:** Should be 1
- **Integer overflow:** Use long long

10. Practice Problems

Easy

Problem	Platform	Main Idea
Small Factorial	Codechef	Simple modulo

Medium

Problem	Platform	Main Idea
$nCr \bmod p$	GFG	Precompute + Fermat

11. Interview Explanation

"I precompute factorials modulo M in $O(n)$ using $\text{fact}[i] = \text{fact}[i-1] \times i \% M$. For inverse factorials, I compute the last one using Fermat's theorem, then fill backwards. This gives $O(1)$ nCr queries via $\text{fact}[n] \times \text{invFact}[k] \times \text{invFact}[n-k]$."

12. Revision Notes

- $\text{fact}[0] = 1, \text{fact}[i] = \text{fact}[i-1] \times i \% \text{MOD}$
- $nCr = \text{fact}[n] \times \text{invFact}[k] \% \text{MOD} \times \text{invFact}[n-k] \% \text{MOD}$
- $n \geq \text{MOD}$ (prime) $\rightarrow$ need Lucas Theorem

13. Final Cheat Sheet

Aspect	Details
Precompute	$O(n)$ factorials, $O(n + \log \text{MOD})$ invFactor
nCr formula	$\text{fact}[n] * \text{invFact}[k] * \text{invFact}[n-k]$
Key code	$\text{fact}[i] = \text{fact}[i-1] * i \% \text{MOD}$
Edge	$n \geq \text{MOD} \rightarrow$ Lucas theorem

9. nCr Basics

1. Overview

Number of ways to choose r items from n distinct items (order doesn't matter). Formula: $C(n, r) = n! / (r! \times (n-r)!)$.

2. Intuition

Analogy: You have n different fruits and want to pick r for a basket. First arrange all n : $n!$ ways. The first r (selected) can be rearranged: divide by $r!$. The last $n-r$ (not selected): divide by $(n-r)!$.

Key properties:

- $C(n, r) = C(n, n-r)$ — symmetry
- $C(n, 0) = C(n, n) = 1$
- $C(n, k) = C(n-1, k-1) + C(n-1, k)$ — Pascal's identity
- $\sum C(n, k) = 2^n$

3. When to Use It

- Counting combinations (order doesn't matter)
- Probability problems
- Pascal's triangle construction
- DP where state depends on choosing/not choosing

4. When Not to Use It

- When order matters (use nPr)
- When items are identical (use stars and bars: $C(n+r-1, r-1)$)

5. Core Concepts

Pascal's Identity

$C(n, k) = C(n-1, k-1) + C(n-1, k)$ — allows DP without factorial.

Symmetry

Always compute with $r = \min(r, n-r)$.

6. Step-by-Step Algorithm

Using factorial:

1. Precompute `fact[0..n]` and `invFact[0..n]`.
2. Return `fact[n] * invFact[r] % MOD * invFact[n-r] % MOD`.

Using Pascal:

1. 2D array `C[n+1][n+1]`.
2. `C[i][0] = C[i][i] = 1`.
3. `C[i][j] = C[i-1][j-1] + C[i-1][j]`.

7. C++ Implementation

```
// Pascal's triangle (space-optimized)
ll nCrPascal(int n, int k) {
    if (k < 0 || k > n) return 0;
    if (k > n - k) k = n - k;
    vector<ll> C(k + 1, 0);
    C[0] = 1;
    for (int i = 1; i <= n; i++) {
        for (int j = min(i, k); j > 0; j--) {
            C[j] = (C[j] + C[j - 1]) % MOD;
        }
    }
    return C[k];
}
```

8. Complexity Analysis

Method	Preprocessing	Per Query	Space
Factorial + inverse	$O(\text{MAXN})$	$O(1)$	$O(\text{MAXN})$
Pascal's triangle	$O(n^2)$	$O(1)$	$O(k)$

9. Common Mistakes

- **Not checking $r > n$:** Must return 0
- **Forgetting symmetry:** Compute with $\min(r, n-r)$
- **Wrong formula:** $nCr = n! / (r! \times (n-r)!)$, NOT $n! / r!$

10. Practice Problems

Easy

Problem	Platform	Main Idea
Unique Paths	LeetCode 62	DP = $C(m+n-2, m-1)$

Medium

Problem	Platform	Main Idea
Combinations	LeetCode 77	Generate all combos

11. Interview Explanation

"nCr counts combinations — selecting r items from n distinct items where order doesn't matter. I compute it using $n! / (r! \times (n-r)!)$. For modulo, I precompute factorials and inverse factorials. I always use symmetry $C(n, r) = C(n, n-r)$ to minimize computation."

12. Revision Notes

- **Formula:** $C(n, r) = n! / (r! \times (n-r)!)$
- **Symmetry:** $C(n, r) = C(n, n-r)$
- **Pascal:** $C(n, k) = C(n-1, k-1) + C(n-1, k)$
- **Key identity:** $\sum C(n, k) = 2^n$

13. Final Cheat Sheet

Aspect	Details
Formula	$n! / (r! \times (n-r)!)$
Symmetry	$C(n, r) = C(n, n-r)$
Precompute	$O(\text{MAXN})$ fact + invFact
Pascal DP	$O(n^2)$ no modulo restrictions

10. Divisors

1. Overview

Divisors of n are integers that divide n without remainder. Divisors come in pairs $(d, n/d)$ where one $\leq \sqrt{n}$, the other $\geq \sqrt{n}$.

2. Intuition

Analogy: Tiling a rectangle of area n . Any valid tile pattern has dimensions $(d, n/d)$. We only need to check widths up to $\sqrt{n}$.

3. When to Use It

- Finding all divisors of a number

- Sum of divisors, count of divisors
- Perfect number checking

4. When Not to Use It

- For prime checking (just check up to $\sqrt{n}$)
- For extremely large n ($10^{12}+$) — $O(\sqrt{n})$ may be borderline

5. Core Concepts

Divisor Pairs

If $n = a \times b$ and $a \leq b$, then $a \leq \sqrt{n}$ and $b \geq \sqrt{n}$.

Count Formula

If $n = p_1^{a_1} \times \dots \times p_k^{a_k}$, then $d(n) = (a_1+1) \times \dots \times (a_k+1)$.

6. Step-by-Step Algorithm

1. Iterate i from 1 to $\sqrt{n}$.
2. If i divides n :
 - Add i .
 - If $i \neq n/i$, add n/i .
3. Sort if needed.

7. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

vector<int> getDivisorsSorted(int n) {
    vector<int> small, large;
    for (int i = 1; i * i <= n; i++) {
        if (n % i == 0) {
            small.push_back(i);
            if (i != n / i) large.push_back(n / i);
        }
    }
    reverse(large.begin(), large.end());
    small.insert(small.end(), large.begin(), large.end());
    return small;
}

long long countDivisors(long long n) {
    long long cnt = 1;
    for (long long p = 2; p * p <= n; p++) {
        if (n % p == 0) {
            long long exp = 0;
            while (n % p == 0) { n /= p; exp++; }
            cnt *= (exp + 1);
        }
    }
    return cnt;
}
```

```

    }
}
if (n > 1) cnt *= 2;
return cnt;
}

```

8. Complexity Analysis

Operation	Time Complexity	Space
Find all divisors	$O(\sqrt{n})$	$O(d(n))$
Count divisors (via factors)	$O(\sqrt{n})$	$O(1)$

9. Common Mistakes

- **Perfect square:** Don't add middle divisor twice
- **Sorted output:** Many problems expect sorted order

10. Practice Problems

Easy

Problem	Platform	Main Idea
Perfect Number	LeetCode 507	Sum of divisors

Hard

Problem	Platform	Main Idea
Common Divisors	Codeforces	Largest divisor dividing all array elements

11. Interview Explanation

"To find all divisors, I iterate from 1 to $\sqrt{n}$. When i divides n , I add both i and n/i . For counting, I prime-factorize and use $d(n) = \prod (a_i + 1)$."

12. Revision Notes

- **Find:** Iterate $i=1$ to $\sqrt{n}$, if $n\%i==0$, add i and n/i
- **Count:** $d(n) = \prod (a_i + 1)$
- **Edge:** $d(1) = 1$

13. Final Cheat Sheet

Aspect	Details
Algorithm	Up to $\sqrt{n}$, collect pairs

Aspect	Details
Complexity	$O(\sqrt{n})$
Count formula	$\prod (a_i + 1)$ from prime factorization

11. Prime Factorization

1. Overview

Express a number as a product of its prime factors. Every integer > 1 has a unique prime factorization (Fundamental Theorem of Arithmetic).

2. Intuition

Analogy: Think of a number as a LEGO structure. Prime numbers are the individual LEGO bricks. Prime factorization is taking the structure apart brick by brick until only indivisible bricks remain.

3. When to Use It

- Finding prime factors of a number
- Computing GCD/LCM using prime exponents
- Computing divisor count/sum
- Cryptography (RSA)

4. When Not to Use It

- For very large numbers ($10^{18}+$) — use Pollard Rho
- When you just need GCD/LCM (Euclidean is faster)

5. Core Concepts

Smallest Prime Factor (SPF)

Precompute SPF via sieve for all numbers up to N , then factor any number in $O(\log n)$.

Uniqueness

Prime factorization is unique — this makes it a powerful tool.

6. Step-by-Step Algorithm

Trial division:

1. For $p = 2$; $p * p \leq n$; $p++$:
 - If p divides n : count exponent, divide n repeatedly.
2. If $n > 1$: store $(n, 1)$ as remaining prime factor.

7. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

using ll = long long;

vector<pair<ll, int>> factorize(ll n) {
    vector<pair<ll, int>> factors;
    for (ll p = 2; p * p <= n; p++) {
        if (n % p == 0) {
            int exp = 0;
            while (n % p == 0) { n /= p; exp++; }
            factors.push_back({p, exp});
        }
    }
    if (n > 1) factors.push_back({n, 1});
    return factors;
}

// SPF-based
const int MAXN = 1000000;
int spf[MAXN + 1];

void sieveSPF() {
    for (int i = 1; i <= MAXN; i++) spf[i] = i;
    for (int p = 2; p * p <= MAXN; p++) {
        if (spf[p] == p) {
            for (int i = p * p; i <= MAXN; i += p) {
                if (spf[i] == i) spf[i] = p;
            }
        }
    }
}

vector<pair<int, int>> factorizeSPF(int n) {
    vector<pair<int, int>> factors;
    while (n > 1) {
        int p = spf[n], exp = 0;
        while (n % p == 0) { n /= p; exp++; }
        factors.push_back({p, exp});
    }
    return factors;
}

```

8. Complexity Analysis

Method	Preprocess	Per Query	Space
Trial division	None	$O(\sqrt{n})$	$O(1)$
SPF with sieve	$O(N \log \log N)$	$O(\log n)$	$O(N)$

9. Common Mistakes

- **Missing remaining factor:** If $n > 1$ after loop, it's a prime factor
- **Inefficient loop:** Check 2 separately, then odd numbers only

10. Edge Cases

Input	Factorization	Notes
1	(empty)	No prime factors
2	[(2, 1)]	Smallest prime

11. Practice Problems

Easy

Problem	Platform	Main Idea
Prime Factorization	GFG	Basic

Hard

Problem	Platform	Main Idea
Largest Prime Factor	Project Euler 3	Find largest

12. Interview Explanation

"Prime factorization expresses a number as a product of primes. The basic method divides by numbers from 2 to $\sqrt{n}$, counting exponents. If $n > 1$ after the loop, it's a prime factor. For multiple queries, I precompute smallest prime factors using a modified sieve, giving $O(\log n)$ factorization per query."

13. Revision Notes

- **Algorithm:** Divide by p from 2 to $\sqrt{n}$ while $p|n$
- **Remaining:** If $n > 1$, it's a prime factor
- **SPF:** Precompute smallest prime factor for $O(\log n)$ factorization
- **Formula:** $n = \prod p_i^{a_i}$

14. Final Cheat Sheet

Aspect	Details
Algorithm	Divide by 2.. $\sqrt{n}$, collect factors
Complexity	$O(\sqrt{n})$ basic, $O(\log n)$ with SPF
Key code	<code>while(n%p==0) { n/=p; exp++; }</code>
Remaining factor	If $n > 1$, it's a prime

12. Modular Inverse

1. Overview

The modular inverse of a modulo M is x such that $(a \times x) \% M = 1$. Exists only if $\gcd(a, M) = 1$.

2. Intuition

In regular arithmetic, dividing by a is multiplying by $1/a$. In modular arithmetic, we find the modular inverse — a number x such that $a \times x \equiv 1 \pmod{M}$. Then $b / a \equiv b \times x \pmod{M}$.

Analogy: On a clock (mod 12), inverse of 5: $(5 \times x) \bmod 12 = 1 \rightarrow x = 5$ because $5 \times 5 = 25 \equiv 1 \pmod{12}$.

3. When to Use It

- Division in modular arithmetic
- Computing nCr , nPr modulo M
- Solving linear congruences
- Rolling hash inversion

4. When Not to Use It

- When $\gcd(a, M) \neq 1$ (inverse doesn't exist)
- For simple addition/subtraction

5. Core Concepts

Fermat's Little Theorem (M prime)

$$a^{(M-1)} \equiv 1 \pmod{M} \rightarrow a^{-1} \equiv a^{(M-2)} \pmod{M}.$$

Extended Euclidean (any M)

Solves $a \times x + M \times y = \gcd(a, M)$. If $\gcd = 1$, x is the inverse.

6. Step-by-Step Algorithm

Using Fermat:

1. Compute $a^{(M-2)} \% M$.

Using Extended Euclidean:

1. Get (g, x, y) from `extendedGcd(a, M)`.
2. If $g \neq 1$: inverse doesn't exist.
3. Return $(x \% M + M) \% M$.

7. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

using ll = long long;

tuple<ll, ll, ll> extendedGcd(ll a, ll b) {
    if (b == 0) return {a, 1, 0};
    auto [g, x1, y1] = extendedGcd(b, a % b);
    return {g, y1, x1 - (a / b) * y1};
}

ll modInv(ll a, ll M) {
    auto [g, x, y] = extendedGcd(a, M);
    if (g != 1) return -1;
    return (x % M + M) % M;
}

ll modInvFermat(ll a, ll M) {
    ll res = 1, b = M - 2; a %= M;
    while (b) { if (b & 1) res = res * a % M; a = a * a % M; b >>= 1; }
    return res;
}

// Precompute inverses of 1..n in O(n)
vector<ll> precomputeInverses(int n, ll M) {
    vector<ll> inv(n + 1);
    inv[1] = 1;
    for (int i = 2; i <= n; i++) {
        inv[i] = M - (M / i) * inv[M % i] % M;
    }
    return inv;
}

```

8. Complexity Analysis

Method	Time	Space	Condition
Extended Euclidean	$O(\log M)$	$O(1)$	Any M ($\gcd=1$)
Fermat (modPow)	$O(\log M)$	$O(1)$	M prime
Precompute 1..n	$O(n)$	$O(n)$	M prime

9. Common Mistakes

- **Using Fermat when M is not prime:** Use Extended Euclidean
- **Forgetting gcd check:** If $\gcd \neq 1$, inverse doesn't exist
- **Negative value:** Normalize with $(x \% M + M) \% M$

10. Edge Cases

Input	Expected	Reason
$\text{inv}(1, M)$	1	$1 \times 1 = 1 \bmod M$
$\text{inv}(M-1, M)$	$M-1$	$(M-1)^2 = 1 \bmod M$
$\text{inv}(0, M)$	Not exist	0 has no inverse

11. Practice Problems

Easy

Problem	Platform	Main Idea
Modular Inverse	GFG	Basic inverse

Medium

Problem	Platform	Main Idea
nCr with inverse	Codeforces	Factorial + inverse

12. Interview Explanation

"The modular inverse of a modulo M is x such that $a \times x \equiv 1 \pmod{M}$. It exists only when $\gcd(a, M) = 1$. I use either Fermat's theorem ($a^{M-2} \bmod M$) when M is prime, or Extended Euclidean for any M . The Extended Euclidean solves $a \times x + M \times y = \gcd(a, M)$ and returns x as the inverse when $\gcd = 1$."

13. Revision Notes

- **Fermat:** $a^{-1} = a^{M-2} \bmod M$ (M prime)
- **Extended Euclid:** $ax + My = 1 \rightarrow x$ is inverse
- **Check gcd:** Must be 1
- **Linear precompute:** $\text{inv}[i] = M - (M/i) \times \text{inv}[M\%i] \% M$

14. Final Cheat Sheet

Aspect	Details
Condition	$\gcd(a, M) = 1$
Fermat	$\text{pow}(a, M-2, M)$ (M prime)
Extended Euclid	$ax + My = 1 \rightarrow x$ is inverse
Precompute	$O(n)$ for inverses of $1..n$
Complexity	$O(\log M)$ per query

13. Fermat's Little Theorem

1. Overview

If p is prime and a is not divisible by p , then $a^{(p-1)} \equiv 1 \pmod{p}$.

2. Intuition

Core idea: For prime p , raising any non-zero residue to $(p-1)$ gives 1 modulo p .

Consequence: $a^{(p-2)} \equiv a^{-1} \pmod{p}$ — used for modular inverse.

Why it works: The numbers $\{1, 2, \dots, p-1\}$ form a multiplicative group. Multiplying each by a rearranges them, so the product of all equals $a^{(p-1)}$ times the product, giving $a^{(p-1)} \equiv 1$.

3. When to Use It

- Computing modular inverse when M is prime
- In primality testing (Fermat primality test)
- Theoretical number theory proofs

4. When Not to Use It

- When M is not prime (use Euler's theorem or Extended Euclidean)
- As a deterministic primality test (pseudoprimes exist)

5. Core Concepts

Application: Modular Inverse

If M is prime, $a^{-1} \equiv a^{(M-2)} \pmod{M}$. Use binary exponentiation.

Application: Reduction of Exponents

$a^b \pmod{M} = a^{(b \bmod (M-1))} \pmod{M}$ (when a and M are coprime).

Carmichael Numbers

Numbers that satisfy $a^{(n-1)} \equiv 1$ for all coprime a but are composite. Deceives the Fermat primality test.

6. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

ll modPow(ll a, ll b, ll M) {
    ll res = 1; a %= M;
    while (b) { if (b & 1) res = res * a % M; a = a * a % M; b >>= 1; }
    return res;
}
```

```

11 modInvFermat(11 a, 11 M) {
    return modPow(a, M - 2, M);
}

int main() {
    cout << modInvFermat(3, 7) << "\n"; // 5 (3-1 mod 7 = 5)
    cout << modPow(3, (13 % 6), 7) << "\n"; // reduce exponent mod (7-1)=6
    return 0;
}

```

7. Complexity Analysis

Operation	Time	Space
Modular inverse (Fermat)	$O(\log M)$	$O(1)$
Exponent reduction	$O(\log M)$	$O(1)$

8. Common Mistakes

- **Using Fermat when M is not prime:** The theorem does NOT hold for composite modulus
- **Forgetting gcd condition:** If a is divisible by p, $a^{(p-1)} \bmod p = 0$, not 1
- **Confusing Fermat's Little Theorem with Fermat's Last Theorem:** Completely different

9. Edge Cases

Input	Issue
$a \% p == 0$	$a^{(p-1)} \equiv 0$, not 1
p not prime	Theorem fails — use Euler's theorem

10. Related Algorithms

- **Euler's theorem:** $a^{\phi(n)} \equiv 1 \pmod n$ for coprime a, n — generalization of Fermat
- **Extended Euclidean:** Alternative for modular inverse, works for any coprime numbers
- **Miller-Rabin:** Uses Fermat's theorem as basis for primality testing

11. Practice Problems

Easy

Problem	Platform	Main Idea
Modular Inverse	GFG	$a^{(M-2)} \bmod M$

Medium

Problem	Platform	Main Idea
nCr mod p	Codeforces	Using Fermat for inverse factorial

12. Interview Explanation

"Fermat's Little Theorem states that for prime p and a not divisible by p , $a^{(p-1)} \equiv 1 \pmod{p}$. This directly gives us a modular inverse: $a^{-1} \equiv a^{(p-2)} \pmod{p}$. I use binary exponentiation to compute this in $O(\log p)$. It's one of the most used theorems in competitive programming for division modulo a prime."

13. Revision Notes

- $a^{(p-1)} \equiv 1 \pmod{p}$ for prime p , $\gcd(a, p) = 1$
- $a^{-1} \equiv a^{(p-2)} \pmod{p}$ — modular inverse
- Exponent reduction: $a^b \pmod{p} = a^{(b \pmod{p-1})} \pmod{p}$
- **Condition:** p must be prime, a not divisible by p

14. Final Cheat Sheet

Aspect	Details
Theorem	$a^{(p-1)} \equiv 1 \pmod{p}$ for prime p
Inverse formula	$a^{-1} = a^{(p-2)} \pmod{p}$
Exp reduction	$a^b \pmod{p} = a^{(b \pmod{p-1})} \pmod{p}$
Condition	p prime, a not multiple of p
Complexity	$O(\log p)$ via binary exponentiation

14. Extended Euclidean Algorithm

1. Overview

Extended Euclidean Algorithm finds integers x, y such that $a \times x + b \times y = \gcd(a, b)$. These coefficients allow computing modular inverses and solving linear Diophantine equations.

2. Intuition

The standard Euclidean gives us $\gcd$. The Extended version also tracks back the coefficients to express $\gcd$ as a linear combination of the original numbers.

Analogy: You know $\gcd(a, b) = g$. Now you want to find two numbers x and y such that $a \times x + b \times y = g$. This is like reverse-engineering the modulo operations we did in the Euclidean algorithm.

Why it works: The Euclidean algorithm steps are reversible. If we know $b \times x_1 + (a \% b) \times y_1 = g$, we can work backwards to get coefficients for a and b .

3. When to Use It

- Computing modular inverse when M is NOT prime
- Solving linear Diophantine equations ($ax + by = c$)
- Finding solutions to Chinese Remainder Theorem
- Breaking RSA in some contexts (factorization with known ϕ)
- Finding integer solutions to $ax \equiv b \pmod{M}$

4. When Not to Use It

- When M is prime and you just need inverse (Fermat is simpler)
- When $\gcd(a, b) \neq 1$ and you need modular inverse (inverse doesn't exist)
- When you only need gcd (standard Euclidean is sufficient)

5. Core Concepts

Linear Diophantine Equation

Equation $ax + by = c$ has integer solutions iff $\gcd(a, b) \mid c$.

Back Substitution

After Euclidean algorithm gives $r_1 = a - q_1 \times b$, $r_2 = b - q_2 \times r_1$, ..., we substitute backwards to express gcd as $a \times x + b \times y$.

Recursive Coefficients

If $\gcd(b, a \% b) = b \times x_1 + (a \% b) \times y_1$, then: $\gcd(a, b) = a \times y_1 + b \times (x_1 - (a // b) \times y_1)$.

6. Step-by-Step Algorithm

1. If $b == 0$, return $(a, 1, 0)$.
2. Recursively get $(g, x_1, y_1) = \text{extendedGcd}(b, a \% b)$.
3. Return $(g, y_1, x_1 - (a // b) \times y_1)$.

7. Dry Run

Find x, y for $30x + 24y = \gcd(30, 24)$.

Level	a	b	a % b	q = a//b	Returns (g, x, y)
1	30	24	6	1	$(6, y_1, x_1 - 1 \times y_1)$
2	24	6	0	4	$(6, y_2, x_2 - 4 \times y_2)$
3	6	0	—	—	$(6, 1, 0)$

Back-substitute:

- Level 3: $g=6, x=1, y=0$
- Level 2: $x_2 = 0, y_2 = 1$. So returns $(6, 0, 1 - 4 \times 0) = (6, 0, 1)$. But wait — let me recalculate.

Actually, from level 3: (g=6, x=1, y=0) Level 2 (a=24, b=6, q=4): $x = 0, y = 1 - 4 \times 0 = 1$. Returns (6, 0, 1). Level 1 (a=30, b=24, q=1): $x = 1, y = 0 - 1 \times 1 = -1$. Returns (6, 1, -1).

Check: $30 \times 1 + 24 \times (-1) = 30 - 24 = 6 = \gcd(30, 24)$. ✓

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

// Returns (g, x, y) such that a*x + b*y = g = gcd(a,b)
tuple<ll, ll, ll> extendedGcd(ll a, ll b) {
    if (b == 0) return {a, 1, 0};
    auto [g, x1, y1] = extendedGcd(b, a % b);
    return {g, y1, x1 - (a / b) * y1};
}

// Modular inverse for any M (if gcd(a, M) = 1)
ll modInv(ll a, ll M) {
    auto [g, x, y] = extendedGcd(a, M);
    if (g != 1) return -1; // no inverse
    return (x % M + M) % M;
}

// Solve linear Diophantine: a*x + b*y = c
// Returns {x, y} or indicates no solution
bool solveDiophantine(ll a, ll b, ll c, ll& x, ll& y) {
    auto [g, x0, y0] = extendedGcd(abs(a), abs(b));
    if (c % g != 0) return false; // no solution

    x = x0 * (c / g);
    y = y0 * (c / g);
    if (a < 0) x = -x;
    if (b < 0) y = -y;

    // General solution: x = x0 + (b/g) * t, y = y0 - (a/g) * t
    return true;
}

int main() {
    auto [g, x, y] = extendedGcd(30, 24);
    cout << g << " " << x << " " << y << "\n"; // 6 1 -1

    cout << modInv(3, 7) << "\n"; // 5
    cout << modInv(4, 10) << "\n"; // -1 (no inverse, gcd=2)

    ll xSol, ySol;
    if (solveDiophantine(30, 24, 12, xSol, ySol)) {
        cout << xSol << " " << ySol << "\n"; // one solution
    }
}
```

```

    return 0;
}

```

9. Python Implementation

```

def extended_gcd(a: int, b: int):
    """Return (g, x, y) such that a*x + b*y = g"""
    if b == 0:
        return (a, 1, 0)
    g, x1, y1 = extended_gcd(b, a % b)
    return (g, y1, x1 - (a // b) * y1)

def mod_inv(a: int, M: int) -> int:
    g, x, y = extended_gcd(a, M)
    if g != 1:
        return -1
    return x % M

def solve_diophantine(a: int, b: int, c: int):
    """Return (x, y) solution to a*x + b*y = c or None"""
    g, x0, y0 = extended_gcd(abs(a), abs(b))
    if c % g != 0:
        return None
    x = x0 * (c // g)
    y = y0 * (c // g)
    if a < 0: x = -x
    if b < 0: y = -y
    return (x, y)

print(extended_gcd(30, 24)) # (6, 1, -1)
print(mod_inv(3, 7))       # 5
print(solve_diophantine(30, 24, 12)) # (2, -2)

```

10. Code Explanation

- **Base case:** $b = 0 \rightarrow \text{gcd} = a$, coefficients: $a \times 1 + 0 \times 0 = a$.
- **Recursive step:** We know $b \times x_1 + (a \% b) \times y_1 = g$. Since $a \% b = a - (a // b) \times b$, we substitute: $g = b \times x_1 + (a - (a // b) \times b) \times y_1$
 $g = a \times y_1 + b \times (x_1 - (a // b) \times y_1)$
 So $x = y_1$, $y = x_1 - (a // b) \times y_1$.
- **Diophantine solver:** After getting one solution (x_0, y_0) for $a \times x + b \times y = g$, scale by c/g to get solution for $a \times x + b \times y = c$.

11. Complexity Analysis

Operation	Time Complexity	Space Complexity
Extended Euclidean	$O(\log \min(a, b))$	$O(\log \min(a, b))$ (recursion) or $O(1)$ (iterative)
Modular inverse via EEA	$O(\log M)$	$O(1)$

12. Common Patterns

Pattern	How to Identify	Approach
Modular inverse (non-prime)	"find $a^{-1} \bmod M$ where M is not prime"	Extended Euclidean
Simple linear congruence	" $ax \equiv b \pmod{M}$ "	$x = b \times a^{-1} \bmod M$ (a^{-1} via EEA)
Two-variable linear eqn	"find integer solutions to $ax + by = c$ "	EEA + scaling by c/g
CRT computation	"combine two congruences"	EEA used to find combination coefficients

13. Common Mistakes

- **Ignoring negative values:** Extended Euclidean works with positive inputs. For negatives, take abs, then adjust sign of result.
- **Confusing x and y:** The recursive relation swaps coefficients. Track carefully.
- **Not checking gcd condition:** Diophantine equation $ax + by = c$ has solution only if $\gcd(a, b) \mid c$.
- **Forgetting to normalize:** Modular inverse can be negative. Always normalize: $(x \% M + M) \% M$.

14. Edge Cases

Input	Expected	Reason
extendedGcd(0, 5)	(5, 0, 1)	$0 \times 0 + 5 \times 1 = 5 = \gcd$
extendedGcd(5, 0)	(5, 1, 0)	$5 \times 1 + 0 \times 0 = 5 = \gcd$
extendedGcd(0, 0)	(0, 1, 0)	$\gcd(0, 0) = 0$ by convention
modInv(4, 10)	-1	$\gcd(4, 10) = 2 \neq 1$
solveDiophantine(2, 4, 3)	No solution	$\gcd(2, 4) = 2 \nmid 3$

15. Variations

Iterative Extended Euclidean

Avoids recursion stack. Uses iterative swapping of coefficients.

Modular Inverse for Large Numbers

Extended Euclidean works for arbitrarily large numbers (as long as arithmetic is supported).

Extended Euclidean for Polynomials

Same algorithm adapted for polynomial GCD, used in coding theory.

16. Related Algorithms

- **Standard Euclidean** — gives only gcd, not coefficients
- **Fermat's Little Theorem** — simpler inverse but requires prime modulus
- **Chinese Remainder Theorem** — uses extended Euclidean to combine congruences
- **Continued Fractions** — extended Euclidean can generate continued fraction expansion

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Modular Inverse	GFG	Direct EEA for inverse	Easy
GCD and Coeffs	Codeforces	Find x, y for $ax + by = g$	Easy

Medium

Problem	Platform	Main Idea	Difficulty
Linear Diophantine	Codeforces 1096C	Solve $ax + by = c$	Medium
Number of Solutions	CSES	Diophantine with range constraints	Medium

Hard

Problem	Platform	Main Idea	Difficulty
GCD of an Array	Codeforces	Extended Euclidean with segment tree	Hard
RSA cryptosystem	LeetCode (variant)	Find d using extended Euclidean	Hard

18. Interview Explanation

"The Extended Euclidean Algorithm not only finds $\text{gcd}(a, b)$ but also coefficients x, y such that $ax + by = \text{gcd}(a, b)$. I implement it recursively — when $b = 0$, return $(a, 1, 0)$. Otherwise recurse with $(b, a \% b)$ and compute new coefficients as $(y_1, x_1 - (a//b) \times y_1)$. This gives me modular inverses for any modulus (not just prime) and lets me solve linear Diophantine equations $ax + by = c$, which has solutions only when $\text{gcd}(a, b)$ divides c ."

19. Revision Notes

- **Output:** (g, x, y) where $ax + by = g = \text{gcd}(a, b)$
- **Recurrence:** $\text{extendedGcd}(a, b) \rightarrow \text{let } (g, x_1, y_1) = \text{extendedGcd}(b, a \% b); \text{return } (g, y_1, x_1 - (a//b) \times y_1)$
- **Base:** $b = 0 \rightarrow (a, 1, 0)$

- **Modular inverse:** $a^{-1} \bmod M = (x \% M + M) \% M$ (gcd must be 1)
- **Diophantine:** $ax + by = c$ has solution iff $g \mid c$. Particular: $x = x_0 \times c/g$.
- **General solution:** $x = x_0 + (b/g) \times t$, $y = y_0 - (a/g) \times t$

20. Final Cheat Sheet

Aspect	Details
Input	a, b (integers)
Output	(g, x, y) where $a \times x + b \times y = g = \gcd(a, b)$
Complexity	$O(\log \min(a, b))$
Key recurrence	$x = y_1, y = x_1 - (a//b) \times y_1$
Mod inverse	$(x \% M + M) \% M$ (when $g = 1$)
Diophantine	Solution exists iff $g \mid c$, scale by c/g
General solution	$x = x_0 + (b/g) \times t, y = y_0 - (a/g) \times t$

15. Chinese Remainder Theorem (CRT)

1. Overview

CRT solves systems of congruences with pairwise coprime moduli. Given:

- $x \equiv a_1 \pmod{m_1}$
- $x \equiv a_2 \pmod{m_2}$
- ...
- $x \equiv a_k \pmod{m_k}$

Find x modulo $M = m_1 \times m_2 \times \dots \times m_k$.

2. Intuition

Analogy: You have k clocks of different sizes ($m_1, m_2, \dots, m_k$). Each shows a different time ($a_1, a_2, \dots, a_k$). CRT tells you the total time elapsed since all clocks were synchronized.

Why it works: If moduli are coprime, there exists a unique solution modulo M . We can combine two congruences at a time using modular inverses.

Step-by-step reasoning:

1. For two congruences $x \equiv a_1 \pmod{m_1}$ and $x \equiv a_2 \pmod{m_2}$:
2. Write $x = a_1 + m_1 \times t$.
3. Substitute: $a_1 + m_1 \times t \equiv a_2 \pmod{m_2} \rightarrow m_1 \times t \equiv a_2 - a_1 \pmod{m_2}$.
4. If $\gcd(m_1, m_2) = 1$, compute $m_1^{-1} \bmod m_2$ and get $t = (a_2 - a_1) \times m_1^{-1} \bmod m_2$.
5. Then $x = a_1 + m_1 \times t \bmod (m_1 \times m_2)$.

3. When to Use It

- Systems of congruences with coprime moduli
- Solving problems involving modular arithmetic with multiple moduli
- Garner's algorithm for large integer representation (big integers via small moduli)
- RSA decryption (CRT speeds up computation)
- Problems that break a large problem into independent modulo constraints

4. When Not to Use It

- When moduli are not pairwise coprime (need generalized CRT)
- When you only have a single congruence (Chinese remainder $\neq$ single remainder problem)
- When large number of moduli (still works but each pairwise combination takes $O(\log m)$)

5. Core Concepts

Pairwise Coprime

All moduli must be pairwise coprime ($\gcd(m_i, m_j) = 1$ for $i \neq j$).

Combined Modulus

The solution is unique modulo $M = m_1 \times m_2 \times \dots \times m_k$.

Incremental Combination

We combine two congruences at a time into one, reducing the system size.

6. Step-by-Step Algorithm

1. Start with $x = a_1, \text{mod} = m_1$.
2. For each next congruence (a_i, m_i) :
 - Let $p = \text{mod}, q = m_i$.
 - Need $x \equiv a \pmod{p}$ and $x \equiv a_i \pmod{q}$.
 - Compute $\text{inv} = \text{modInv}(p \% q, q)$ (inverse of p modulo q , since $\gcd(p, q) = 1$).
 - $t = ((a_i - x) \% q + q) \% q * \text{inv} \% q$.
 - $x = x + p \times t$.
 - $\text{mod} = \text{mod} \times m_i$.
3. Return $(x \% \text{mod} + \text{mod}) \% \text{mod}$.

7. Dry Run

Solve: $x \equiv 2 \pmod{3}, x \equiv 3 \pmod{5}, x \equiv 2 \pmod{7}$.

Step 1: Combine first two.

Variable	Value	Calculation
a_1	2	—
m_1	3	—

Variable	Value	Calculation
a_2	3	—
m_2	5	—
inv	2	$\text{modInv}(3, 5) = 2$ ($3 \times 2 = 6 \equiv 1$)
diff	$(3-2)\%5 = 1$	—
t	$1 \times 2 \% 5 = 2$	—
x	$2 + 3 \times 2 = 8$	—
mod	$3 \times 5 = 15$	—

Check: $8 \equiv 2 \pmod{3}$ ✓, $8 \equiv 3 \pmod{5}$ ✓.

Step 2: Combine with third.

Variable	Value	Calculation
a	8	—
p	15	—
a_3	2	—
m_3	7	—
inv	1	$\text{modInv}(15, 7) = 1$ ($15 \equiv 1 \pmod{7}$, $\text{inv}=1$)
diff	$(2-8)\%7 = 1$	—
t	$1 \times 1 \% 7 = 1$	—
x	$8 + 15 \times 1 = 23$	—
mod	$15 \times 7 = 105$	—

Solution: $x \equiv 23 \pmod{105}$.

Verify: $23 \pmod{3} = 2$ ✓, $23 \pmod{5} = 3$ ✓, $23 \pmod{7} = 2$ ✓.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

tuple<ll, ll, ll> extendedGcd(ll a, ll b) {
    if (b == 0) return {a, 1, 0};
    auto [g, x1, y1] = extendedGcd(b, a % b);
    return {g, y1, x1 - (a / b) * y1};
}
```

```

11 modInv(ll a, ll M) {
    auto [g, x, y] = extendedGcd(a, M);
    if (g != 1) return -1;
    return (x % M + M) % M;
}

// CRT for two congruences:  $x \equiv a_1 \pmod{m_1}$ ,  $x \equiv a_2 \pmod{m_2}$ 
// Returns {x, lcm(m1,m2)} or {-1, -1} if inconsistent
pair<ll, ll> crtPair(ll a1, ll m1, ll a2, ll m2) {
    auto [g, p, q] = extendedGcd(m1, m2);
    if ((a2 - a1) % g != 0) return {-1, -1}; // no solution

    // Combine
    ll lcm = m1 / g * m2;
    ll t = ((a2 - a1) / g) % (m2 / g);
    t = (t * modInv(m1 / g, m2 / g)) % (m2 / g);

    ll x = a1 + m1 * t;
    x = (x % lcm + lcm) % lcm;
    return {x, lcm};
}

// CRT for multiple congruences
11 crt(const vector<ll>& remainders, const vector<ll>& moduli) {
    ll x = remainders[0], M = moduli[0];
    for (int i = 1; i < remainders.size(); i++) {
        auto [newX, newM] = crtPair(x, M, remainders[i], moduli[i]);
        if (newX == -1) return -1; // inconsistent
        x = newX;
        M = newM;
    }
    return x;
}

int main() {
    //  $x \equiv 2 \pmod{3}$ ,  $x \equiv 3 \pmod{5}$ ,  $x \equiv 2 \pmod{7}$ 
    vector<ll> rem = {2, 3, 2};
    vector<ll> mod = {3, 5, 7};

    ll x = crt(rem, mod);
    cout << x << "\n"; // 23

    // Verify
    cout << x % 3 << " " << x % 5 << " " << x % 7 << "\n"; // 2 3 2
    return 0;
}

```

9. Python Implementation

```

def extended_gcd(a: int, b: int):
    if b == 0:
        return (a, 1, 0)
    g, x1, y1 = extended_gcd(b, a % b)
    return (g, y1, x1 - (a // b) * y1)

def mod_inv(a: int, M: int) -> int:
    g, x, y = extended_gcd(a, M)
    if g != 1:
        return -1
    return x % M

def crt(remainders: list, moduli: list) -> int:
    """CRT for multiple congruences with coprime moduli."""
    x = remainders[0]
    M = moduli[0]

    for i in range(1, len(remainders)):
        a1, m1 = x, M
        a2, m2 = remainders[i], moduli[i]

        g, p, q = extended_gcd(m1, m2)
        if (a2 - a1) % g != 0:
            return -1 # inconsistent

        lcm = m1 // g * m2
        t = ((a2 - a1) // g) % (m2 // g)
        inv = mod_inv(m1 // g, m2 // g)
        t = (t * inv) % (m2 // g)

        x = a1 + m1 * t
        x %= lcm
        M = lcm

    return x

# Example
print(crt([2, 3, 2], [3, 5, 7])) # 23

```

10. Code Explanation

- **crtPair**: Combines two congruences (a_1, m_1) and (a_2, m_2) .
 1. Check if solution exists: $(a_2 - a_1) \% g == 0$ where $g = \gcd(m_1, m_2)$.
 2. Compute combined modulus: $\text{lcm} = m_1 / g \times m_2$.
 3. Find t : $t = ((a_2 - a_1) / g) \times (m_1/g)^{-1} \bmod (m_2/g)$.
 4. Solution: $x = a_1 + m_1 \times t \bmod \text{lcm}$.
- **crt (multiple)**: Iteratively combines congruences using crtPair.

11. Complexity Analysis

Operation	Time Complexity	Space
CRT (k congruences)	$O(k \log M)$	$O(1)$
CRT (pair)	$O(\log \max(m_1, m_2))$	$O(1)$

12. Common Patterns

Pattern	How to Identify	Approach
Standard CRT	" $x \equiv a_1 \pmod{m_1}$, $x \equiv a_2 \pmod{m_2}$, ..."	Pairwise combination
Unknown modulus	"find number with given remainders"	CRT to find x
CRT with non-coprime moduli	Moduli share common factors	Use generalized CRT (check g first)
Speed up exponentiation	"RSA decryption"	CRT splits mod p and mod q

13. Common Mistakes

- **Forgetting to check gcd:** If $\text{gcd}(m_1, m_2)$ doesn't divide $(a_2 - a_1)$, no solution exists.
- **Assuming coprime moduli without checking:** CR problem can have non-coprime moduli but needs special handling.
- **Overflow in modulus multiplication:** Product of moduli can overflow 64-bit. Use big integers if needed.
- **Inverse computation:** $\text{modInv}(m_1, m_2)$ only works when m_1 and m_2 are coprime. For generalized CRT, divide by g first.

14. Edge Cases

Input	Expected	Reason
Two identical congruences	Any is fine	Both give same constraint
Inconsistent system	No solution	e.g., $x \equiv 1 \pmod{2}$, $x \equiv 0 \pmod{4}$
Single congruence	$x \equiv a \pmod{m}$	Trivially the given congruence
Large moduli product	Overflow	Use Python or big integer library

15. Variations

Generalized CRT

When moduli are NOT pairwise coprime:

- Check $\text{gcd}(m_1, m_2) \mid (a_2 - a_1)$ for each pair.
- Use the same formula but divide by g at each step.

Garner's Algorithm

Alternative CRT algorithm that avoids large intermediate products by working with mixed-radix representation.

CRT for RSA

In RSA decryption, CRT splits the exponentiation modulo N into two smaller exponentiations modulo p and q , combining results at the end. This gives $\sim 4\times$ speedup.

16. Related Algorithms

- **Extended Euclidean** — used to compute modular inverses within CRT
- **Modular Inverse** — needed for CRT computation
- **Garner's Algorithm** — alternative CRT implementation
- **RSA Algorithm** — practical application of CRT

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Chinese Remainder Theorem	GFG	Basic CRT implementation	Easy
CRT for two equations	Codeforces	Simple two-modulus problem	Easy

Medium

Problem	Platform	Main Idea	Difficulty
Remainder Theorem	SPOJ	Generalized CRT	Medium
B. Chinese Remainder Theorem	Codeforces	CRT applications	Medium

Hard

Problem	Platform	Main Idea	Difficulty
RSA decryption with CRT	LeetCode (variant)	CRT for exponentiation	Hard
Number Theory Problem	Codeforces	CRT + other number theory	Hard

18. Interview Explanation

"CRT solves systems of congruences with pairwise coprime moduli. I combine them incrementally — for two congruences $x \equiv a_1 \pmod{m_1}$ and $x \equiv a_2 \pmod{m_2}$, I write $x = a_1 + m_1 \times t$, substitute into the second equation, and solve for t using modular inverse. The solution is unique modulo the product of the moduli. For the theorem to apply, I need to verify the moduli are pairwise coprime."

19. Revision Notes

- **System:** $x \equiv a_i \pmod{m_i}$, all m_i pairwise coprime
- **Solution:** Unique modulo $M = \prod m_i$
- **Pairwise combination:** $x = a_1 + m_1 \times (a_2 - a_1) \times (m_1^{-1} \pmod{m_2}) \pmod{(m_1 \times m_2)}$
- **Check:** $\gcd(m_1, m_2)$ must divide $(a_2 - a_1)$ for generalized case
- **Applications:** RSA speedup, large integer representation

20. Final Cheat Sheet

Aspect	Details
When to use	System of congruences with coprime moduli
Algorithm	Pairwise combination: $x = a_1 + m_1 \times \text{inv} \times (a_2 - a_1) \pmod{\text{prod}}$
Complexity	$O(k \log M)$ for k congruences
Condition	moduli must be pairwise coprime (or check g)
Key formula	$t = (a_2 - a_1) \times (m_1^{-1} \pmod{m_2}) \pmod{m_2}$
Edge case	Inconsistent moduli $\rightarrow$ no solution

16. Inclusion-Exclusion Principle

1. Overview

Inclusion-Exclusion counts elements in the union of sets by alternating adding and subtracting intersections. For two sets: $|A \cup B| = |A| + |B| - |A \cap B|$. For n sets, we alternate signs based on subset size.

2. Intuition

Analogy: You're counting people who like either pizza or burgers. If you count pizza-lovers + burger-lovers, you've double-counted those who like both. Subtract them once. For three items (pizza, burger, pasta): add singles, subtract pairs, add triple.

General formula: $|A_1 \cup A_2 \cup \dots \cup A_n| = \sum |A_i| - \sum |A_i \cap A_j| + \sum |A_i \cap A_j \cap A_k| - \dots + (-1)^{n+1} |A_1 \cap A_2 \cap \dots \cap A_n|$.

3. When to Use It

- Counting elements that satisfy at least one of several conditions
- Counting numbers divisible by at least one of several numbers
- Counting subsets with certain properties
- Solving derangement problems
- Counting coprime numbers in a range
- Counting distinct positions/arrangements with constraints

4. When Not to Use It

- When sets are disjoint (just add them — no overlap to worry about)
- When n is large (2^n complexity may be prohibitive; use DP or Mobius inversion)

- When individual intersections are hard to compute

5. Core Concepts

Alternating Sum

The sign alternates: + for odd-sized subsets, - for even-sized (or vice versa, depending on formula).

Bitmask Iteration

Iterate over subsets using bitmasks (0 to 2^n-1) to enumerate all combinations.

Derangements

Number of permutations with no fixed points: $D(n) = n! \sum_{i=0}^n (-1)^i / i!$.

6. Step-by-Step Algorithm

1. Identify the sets/conditions.
2. For each non-empty subset of conditions:
 - Determine the size of the intersection of that subset.
 - If subset size is odd: add to result.
 - If subset size is even: subtract from result.

7. Dry Run

Count numbers from 1 to 20 divisible by 2 or 3 or 5.

Subset	Bits	Condition	Count	Sign	Contribution
{2}	001	$\div 2$	10	+	+10
{3}	010	$\div 3$	6	+	+6
{5}	100	$\div 5$	4	+	+4
{2,3}	011	$\div 6$ (lcm)	3	-	-3
{2,5}	101	$\div 10$	2	-	-2
{3,5}	110	$\div 15$	1	-	-1
{2,3,5}	111	$\div 30$	0	+	+0

Total: $10 + 6 + 4 - 3 - 2 - 1 + 0 = 14$.

Verify: Numbers divisible by 2, 3, or 5 in 1..20: {2,3,4,5,6,8,9,10,12,14,15,16,18,20} = 14. ✓

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;
```

```

using ll = long long;

// Count numbers from 1 to N divisible by at least one prime in 'primes'
ll inclusionExclusion(ll N, const vector<ll>& primes) {
    int n = primes.size();
    ll result = 0;

    // Iterate over all non-empty subsets using bitmask
    for (int mask = 1; mask < (1 << n); mask++) {
        ll lcm = 1;
        int bits = 0;
        bool overflow = false;

        for (int i = 0; i < n; i++) {
            if (mask & (1 << i)) {
                bits++;
                // LCM = product if all are coprime (which primes are)
                lcm *= primes[i];
                if (lcm > N) { overflow = true; break; }
            }
        }

        if (overflow) continue;

        ll cnt = N / lcm;
        if (bits % 2 == 1) result += cnt;
        else result -= cnt;
    }
    return result;
}

// Example: calculate Euler Totient using inclusion-exclusion
ll eulerTotientIE(ll n) {
    // Prime factors of n
    vector<ll> factors;
    ll temp = n;
    for (ll p = 2; p * p <= temp; p++) {
        if (temp % p == 0) {
            factors.push_back(p);
            while (temp % p == 0) temp /= p;
        }
    }
    if (temp > 1) factors.push_back(temp);

    // Numbers coprime to n = n - numbers sharing any prime factor
    return n - inclusionExclusion(n, factors);
}

int main() {
    vector<ll> primes = {2, 3, 5};
    cout << inclusionExclusion(20, primes) << "\n"; // 14
    cout << eulerTotientIE(30) << "\n";           // 8
}

```

```
    return 0;
}
```

9. Python Implementation

```
def inclusion_exclusion(N: int, primes: list) -> int:
    """Count numbers from 1 to N divisible by at least one prime factor."""
    n = len(primes)
    result = 0

    for mask in range(1, 1 << n):
        lcm = 1
        bits = 0
        overflow = False

        for i in range(n):
            if mask & (1 << i):
                bits += 1
                lcm *= primes[i]
                if lcm > N:
                    overflow = True
                    break

        if overflow:
            continue

        cnt = N // lcm
        if bits % 2 == 1:
            result += cnt
        else:
            result -= cnt

    return result

print(inclusion_exclusion(20, [2, 3, 5])) # 14
```

10. Code Explanation

- **Bitmask enumeration:** Each bit represents whether a set is included. From 1 to $(1 < n) - 1$, we cover all non-empty subsets.
- **LCM computation:** For coprime primes, the intersection is numbers divisible by their product. For non-coprime conditions, use LCM.
- **Overflow check:** If LCM exceeds N, skip — contribution is 0.
- **Sign:** Odd-sized subsets add (inclusion), even-sized subtract (exclusion).

11. Complexity Analysis

Variant	Time Complexity	Space
Bitmask enumeration	$O(2^n \times n)$	$O(n)$
Recursive	$O(2^n)$	$O(n)$

For $n \leq 20$, $2^n \approx 10^6$ — manageable. For larger n , use DP or Mobius inversion.

12. Common Patterns

Pattern	How to Identify	Approach
Count divisible by any	"numbers divisible by 2 or 3 or 5"	IE over prime factors
Coprime count	"numbers coprime to n in range"	IE = total - sharing any factor
Good strings/sequences	"strings avoiding certain patterns"	Count bad strings via IE
Derangements	"permutations with no fixed points"	$D(n) = n! \sum (-1)^i / i!$
Number of surjections	"onto functions from set A to set B"	IE over missing elements

13. Common Mistakes

- **Wrong starting point:** Bitmasks start from 1 (non-empty subsets), not 0 (empty set).
- **Non-coprime conditions:** For LCM of non-coprime numbers, compute actual LCM, not product.
- **Sign confusion:** Be consistent — either start with positive for size 1, or use formula $(-1)^{(\text{bits}+1)}$.
- **Integer overflow:** Product of many primes can overflow. Check and skip when $\text{LCM} > \text{boundary}$.
- **Missing empty set:** The empty subset represents the entire universe. Usually not needed in IE.

14. Edge Cases

Input	Expected	Reason
$N = 0$	0	No numbers
Empty primes list	0	No conditions
Single prime	N / p	Direct divisibility count
Large n ($n = 30$)	2^{30} iterations	Too slow; use smarter approach

15. Variations

Mobius Inversion

A more powerful technique that can replace IE in many counting problems. Uses the Mobius function (μ) as coefficients.

IE for Probability

Same principle applied to probability: $P(A \cup B) = P(A) + P(B) - P(A \cap B)$.

Principle of Inclusion-Exclusion on Sets

When sets have different structures (not just divisibility), the same alternating sum applies.

16. Related Algorithms

- **Mobius function** — provides coefficients for IE over divisor lattices
- **Principle of Counting** — fundamental counting technique
- **Derangements** — classic IE application
- **Burnside's Lemma** — counts orbits under group action using IE-like reasoning

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Count numbers divisible by 2 or 3	LeetCode (variant)	Direct IE	Easy
Happy Numbers	Codeforces	Simple IE	Easy

Medium

Problem	Platform	Main Idea	Difficulty
Coprime Count	CSES	IE over prime factors	Medium
Count Good Numbers	Codeforces	IE + binary search	Medium

Hard

Problem	Platform	Main Idea	Difficulty
Derangements	Project Euler	Classic IE formula	Hard
Inclusion-Exclusion	Codeforces	Complex IE with multiple conditions	Hard

18. Interview Explanation

"Inclusion-Exclusion counts elements in the union of sets by alternating adding and subtracting intersections. For conditions 2, 3, 5 up to N, I iterate through all $2^n - 1$ subsets using bitmasks. For each subset, I compute the intersection size (numbers divisible by LCM), and add or subtract depending on subset size parity. The complexity is $O(2^n)$, so I use it when $n \leq 20$. For larger n, I'd use Mobius inversion or DP."

19. Revision Notes

- **Formula:** $| \cup A_i | = \sum |A_i| - \sum |A_i \cap A_j| + \sum |A_i \cap A_j \cap A_k| - \dots$
- **Algorithm:** Iterate bitmasks $1..2^n - 1$, count bits, compute intersection, add/subtract
- **Complexity:** $O(2^n)$
- **Sign:** + for odd count, - for even count
- **Intersection:** For divisibility, use LCM of the selected numbers

20. Final Cheat Sheet

Aspect	Details
When to use	Counting elements satisfying at least one condition
Algorithm	Bitmask iteration over all non-empty subsets
Complexity	$O(2^n)$ — use for $n \leq 15-20$
Key formula	<code>result += (-1)^(bits+1) × intersection(mask)</code>
Sign rule	Odd bits → add, Even bits → subtract
For divisibility	Use LCM for intersection, skip if $LCM > N$

17. Pigeonhole Principle

1. Overview

If n items are placed into m boxes and $n > m$, then at least one box contains at least two items. Simple yet powerful idea used in combinatorial proofs and problem-solving.

2. Intuition

Analogy: You have 10 pigeons and 9 holes. When all pigeons enter holes, at least one hole must have 2+ pigeons. There's simply no way to place 10 distinct items into 9 slots with each slot having at most 1 item.

Stronger form: If n items are placed into m boxes, at least one box has $\lceil n/m \rceil$ or more items.

Why it's powerful: The principle itself is obvious, but its applications in competitive programming often require creative problem framing — identifying what the "pigeons" and "holes" are.

3. When to Use It

- Proving existence (that something must happen)
- Problems asking "show that there must be..."
- Problems involving sums/prefixes modulo M (pigeonholes = M remainders)
- Subset sum problems (2^n subsets, $n+1$ possible prefix sums)
- Problems with constraints that force a particular structure
- Counting arguments where a range is limited

4. When Not to Use It

- When you need to construct the actual arrangement (proves existence, not construction)
- When the pigeonholes are too many ($n \leq m$, principle doesn't apply)
- When you need exact counts (pigeonhole only gives lower bounds)

5. Core Concepts

Generalized Pigeonhole

With n items in m boxes, some box has $\geq \lceil n/m \rceil$ items.

Dirichlet's Principle

Another name for the pigeonhole principle.

Typical Application — Prefix Sums

Given n integers, two have sums with same remainder modulo n .

- Compute prefix sums modulo $n \rightarrow n+1$ values for n possible remainders.
- By pigeonhole, at least two prefix sums have same remainder.
- Subarray between them has $\text{sum} \equiv 0 \pmod{n}$.

6. Step-by-Step Application

To apply pigeonhole principle:

1. Identify what the "pigeons" are (items being placed).
2. Identify what the "holes" are (categories/boxes).
3. Count pigeons (n) and holes (m).
4. If $n > m$, conclusion: at least one hole has 2+ items.
5. Derive the problem-specific consequence.

7. Dry Run

Problem: Prove that in any set of n integers, there exists a non-empty subset whose sum is divisible by n .

Setup:

- Pigeons = prefix sums: $S_0 = 0, S_1 = a_1, S_2 = a_1 + a_2, \dots, S_n = a_1 + \dots + a_n$
- Holes = remainders modulo n : $0, 1, 2, \dots, n-1$
- Count: $n+1$ pigeons, n holes $\rightarrow$ at least two prefix sums share a remainder.

Conclusion: If $S_i \equiv S_j \pmod{n}$, then sum of elements from $i+1$ to j is divisible by n .

8. C++ Implementation — Subset with Sum Divisible by n

```
#include <bits/stdc++.h>
using namespace std;

// Find a subarray with sum divisible by n (pigeonhole principle)
pair<int, int> findSubarrayDivisibleByN(vector<int>& arr, int n) {
    // Prefix sum modulo n
    vector<int> pref(n + 1, 0);
    for (int i = 1; i <= n; i++) {
        pref[i] = (pref[i - 1] + arr[i - 1]) % n;
    }

    // Find two prefix sums with same remainder
    vector<int> seen(n, -1); // track first occurrence of each remainder
```

```

    for (int i = 0; i <= n; i++) {
        if (seen[pref[i]] != -1) {
            return {seen[pref[i]] + 1, i}; // 1-indexed
        }
        seen[pref[i]] = i;
    }
    return {-1, -1}; // should never happen
}

int main() {
    vector<int> arr = {3, 1, 7, 2, 8};
    auto [l, r] = findSubarrayDivisibleByN(arr, arr.size());
    cout << "Subarray [" << l << ", " << r << "]: ";
    for (int i = l - 1; i < r; i++) cout << arr[i] << " ";
    cout << "\n";
    return 0;
}

```

9. Python Implementation

```

def find_subarray_divisible_by_n(arr: list):
    n = len(arr)
    pref = [0] * (n + 1)
    for i in range(1, n + 1):
        pref[i] = (pref[i - 1] + arr[i - 1]) % n

    seen = [-1] * n
    for i in range(n + 1):
        if seen[pref[i]] != -1:
            return (seen[pref[i]] + 1, i)
        seen[pref[i]] = i
    return (-1, -1)

arr = [3, 1, 7, 2, 8]
l, r = find_subarray_divisible_by_n(arr)
print(f"Subarray [{l}, {r}]: {arr[l-1:r]}")

```

10. Complexity Analysis

Variant	Time Complexity	Space
Prefix sum with modulo	$O(n)$	$O(n)$
General pigeonhole proof	$O(1)$	$O(1)$

11. Common Patterns

Pattern	How to Identify	Approach
Subset sum divisible by n	Any set of n integers	Prefix sums modulo n
Two equal remainders	"Show two numbers have same property"	Map items to categories
Friends at party	"At least two people know same number"	n people, $n-1$ possible counts
Erdős–Ginzburg–Ziv	$2n-1$ integers, subset of size n with sum divisible by n	Advanced pigeonhole
$N + 1$ numbers from 1 to $2n$	At least one divides another	Use binary representation

12. Common Mistakes

- **Counting pigeons and holes wrong:** Make sure you correctly count both.
- **Assuming the principle works for construction:** It proves existence but doesn't tell you how to find it (though we can often find via direct algorithm as shown above).
- **Pigeonhole when $n = m$:** The principle requires $n > m$. If $n = m$, it's possible each hole has exactly 1 item (no guarantee).
- **Forgetting that the empty set is valid:** In prefix sum problems, $S_0 = 0$ is a valid prefix sum.

13. Edge Cases

Input	Expected	Reason
$n=1$, $arr=[1]$	Subarray $[1,1]$	$1 \div 1 = 0$ remainder
arr with all same numbers	Works	Prefix sums still work
$n=0$	Degenerate	No items to check

14. Variations

Erdős–Ginzburg–Ziv Theorem

From any $2n-1$ integers, there exists a subset of size n whose sum is divisible by n .

Dirichlet's Approximation Theorem

For any real α and integer N , there exist integers p, q with $1 \leq q \leq N$ such that $|q\alpha - p| < 1/N$.

15. Related Problems

Easy

Problem	Platform	Main Idea	Difficulty
Divisible Subarray	GFG	Prefix sums modulo n	Easy
Pigeonhole Principle basics	Codeforces	Simple application	Easy

Medium

Problem	Platform	Main Idea	Difficulty
Subarray Sums Divisible by K	LeetCode 974	Pigeonhole + prefix sums	Medium
Birthday Paradox	Codeforces	Pigeonhole variant	Medium

Hard

Problem	Platform	Main Idea	Difficulty
Erdős–Ginzburg–Ziv	Codeforces	Advanced pigeonhole	Hard
Pigeonhole Principle	Math Olympiad	Creative applications	Hard

16. Interview Explanation

"The pigeonhole principle says if n items go into m boxes and $n > m$, at least one box has 2+ items. In programming, a classic application: given n integers, there's always a non-empty subset whose sum is divisible by n . I prove it by looking at $n+1$ prefix sums modulo n — by pigeonhole, two share a remainder, so the subarray between them has sum divisible by n . The algorithm to find this is $O(n)$."

17. Revision Notes

- **Principle:** n items, m boxes, $n > m \rightarrow$ some box has ≥ 2 items
- **Generalized:** Some box has $\geq \lceil n/m \rceil$ items
- **Classic CP trick:** Prefix sums modulo $n \rightarrow$ at least two equal $\rightarrow$ subarray divisible by n
- **Complexity:** $O(n)$ to find such subarray
- **Key insight:** Identify the right pigeons and holes for each problem

18. Final Cheat Sheet

Aspect	Details
When to use	Existence proofs, forced patterns, modular sums
Core idea	More items than boxes $\rightarrow$ collision
CP application	Prefix sums modulo $n \rightarrow$ subarray sum divisible by n
Complexity	$O(n)$ for algorithmic application
Key code	Track first occurrence of each remainder
Pitfall	Proves existence but not always construction

18. Catalan Numbers

1. Overview

Catalan numbers form a sequence of natural numbers with many combinatorial interpretations. The n -th

Catalan number: $C_n = \frac{1}{(n+1)} \times C(2n, n) = \frac{(2n)!}{(n! \times (n+1)!})$.

First few: 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, ...

2. Intuition

Catalan numbers count objects that can be built recursively by splitting in two.

Analogy: How many ways can you triangulate a convex $(n+2)$ -gon? Or how many valid sequences of n pairs of parentheses? Or how many binary trees with n nodes? All of these are counted by C_n .

Recursive structure: $C_0 = 1$, $C_{n+1} = \sum_{i=0}^n C_i \times C_{n-i}$.

This recurrence captures the "divide into two independent parts" pattern.

3. When to Use It

- Counting valid parentheses expressions with n pairs
- Counting binary trees with n nodes
- Counting triangulations of convex polygon
- Counting monotonic paths not crossing diagonal (Dyck paths)
- Counting ways to connect points on a circle without crossing
- Counting ballot sequences (A always ahead of B)
- Counting non-crossing partitions

4. When Not to Use It

- When order matters and objects are distinct (Catalan counts isomorphic structures)
- When the recurrence doesn't fit the split-into-two pattern
- When n is large and modulo isn't needed ($C(2n, n)$ grows exponentially)

5. Core Concepts

Formula

$$C_n = \frac{C(2n, n)}{(n+1)} = \frac{(2n)!}{(n! \times (n+1)!)}$$

Recurrence

$$C_0 = 1, C_{n+1} = \sum_{i=0}^n C_i \times C_{n-i} \text{ for } n \geq 0.$$

Why the $/(n+1)$ factor

From the total $C(2n, n)$ paths on a grid from $(0,0)$ to $(2n,0)$ with steps $+1$ and -1 , exactly $1/(n+1)$ of them stay above the x -axis (Dyck paths).

Growth

$C_n \sim 4^n / (n^{3/2} \times \sqrt{\pi})$. Exponential growth.

6. Dry Run

Compute C_3 :

Using formula: $C_3 = C(6,3) / 4 = 20 / 4 = 5$.

Recurrence:

- $C_0 = 1$
- $C_1 = C_0 \times C_0 = 1$
- $C_2 = C_0 \times C_1 + C_1 \times C_0 = 1 + 1 = 2$
- $C_3 = C_0 \times C_2 + C_1 \times C_1 + C_2 \times C_0 = 1 \times 2 + 1 \times 1 + 2 \times 1 = 5$

7. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const int MOD = 1e9 + 7;

const int MAXN = 1000000;
ll fact[2 * MAXN + 1], invFact[2 * MAXN + 1];

ll modPow(ll a, ll b, ll mod = MOD) {
    ll res = 1; a %= mod;
    while (b) { if (b & 1) res = res * a % mod; a = a * a % mod; b >>= 1; }
    return res;
}

void precompute(int n) {
    fact[0] = 1;
    for (int i = 1; i <= 2 * n; i++) fact[i] = fact[i - 1] * i % MOD;
    invFact[2 * n] = modPow(fact[2 * n], MOD - 2);
    for (int i = 2 * n; i >= 1; i--) invFact[i - 1] = invFact[i] * i % MOD;
}

ll nCr(ll n, ll r) {
    if (r < 0 || r > n) return 0;
    return fact[n] * invFact[r] % MOD * invFact[n - r] % MOD;
}

ll catalan(int n) {
    //  $C_n = C(2n, n) / (n+1)$ 
    return nCr(2 * n, n) * modPow(n + 1, MOD - 2) % MOD;
}

// DP approach (for small n, no modulo needed)
ll catalanDP(int n) {
    vector<ll> C(n + 1, 0);
    C[0] = 1;
    for (int i = 1; i <= n; i++) {
```

```

        for (int j = 0; j < i; j++) {
            C[i] += C[j] * C[i - j - 1];
        }
    }
    return C[n];
}

int main() {
    precompute(10);
    for (int i = 0; i <= 10; i++) {
        cout << "C[" << i << "] = " << catalan(i) << "\n";
    }
    // 1 1 2 5 14 42 132 429 1430 4862 16796
    return 0;
}

```

9. Python Implementation

```

MOD = 10**9 + 7

def mod_pow(a: int, b: int) -> int:
    res = 1
    a %= MOD
    while b:
        if b & 1: res = res * a % MOD
        a = a * a % MOD
        b >>= 1
    return res

def precompute_fact(n: int):
    fact = [1] * (2 * n + 1)
    for i in range(1, 2 * n + 1):
        fact[i] = fact[i - 1] * i % MOD
    inv_fact = [1] * (2 * n + 1)
    inv_fact[2 * n] = mod_pow(fact[2 * n], MOD - 2)
    for i in range(2 * n, 0, -1):
        inv_fact[i - 1] = inv_fact[i] * i % MOD
    return fact, inv_fact

def nCr(n, r, fact, inv_fact):
    if r < 0 or r > n: return 0
    return fact[n] * inv_fact[r] % MOD * inv_fact[n - r] % MOD

fact, inv_fact = precompute_fact(10)

def catalan(n: int) -> int:
    return nCr(2 * n, n, fact, inv_fact) * mod_pow(n + 1, MOD - 2) % MOD

for i in range(11):
    print(f"C[{i}] = {catalan(i)}")

```

10. Code Explanation

- **Formula approach:** $C_n = C(2n, n) / (n+1) \bmod \text{MOD}$. Compute $C(2n, n)$ using factorials, then multiply by modular inverse of $(n+1)$.
- **DP approach:** $O(n^2)$ recurrence based on the convolution. Not suitable for large n but works for $n \leq 5000$.
- **Precomputation:** Factorials up to $2n$ needed for $C(2n, n)$.

11. Complexity Analysis

Method	Time	Space	Notes
Formula (factorial)	$O(n)$	$O(n)$	Requires modulo
DP	$O(n^2)$	$O(n)$	No modulo needed
Direct product	$O(n)$	$O(1)$	For exact integer

12. Common Patterns

Pattern	How to Identify	Catalan Numbers?
Valid parentheses	"n pairs of parentheses"	C_n
Binary trees	"binary trees with n nodes"	C_n
Dyck paths	"paths from (0,0) to (2n,0) not below x-axis"	C_n
Polygon triangulation	"triangulations of (n+2)-gon"	C_n
Ballot problem	"A always ahead of B in n votes each"	C_n
Non-crossing handshakes	"2n people around a circle"	C_n
Stack permutations	"permutations avoidable with a stack"	C_n

13. Common Mistakes

- **Using the wrong formula:** $C_n = C(2n, n) / (n+1)$, NOT $C(2n, n)$ alone.
- **Off-by-one:** $C_0 = 1$ (empty tree, empty parentheses). Many applications use 1-indexed.
- **Overflow:** $C(2n, n)$ grows very fast. Use modulo or big integers.
- **Forgetting modular inverse:** Division by $(n+1)$ needs modular inverse in mod arithmetic.

14. Edge Cases

n	C_n	Notes
0	1	Base case
1	1	One binary tree with 1 node
2	2	()() and (())
10	16796	Grows fast

15. Variations

Catalan's Triangle

Generalization where $C(n, k)$ counts paths with k up-steps and $n-k$ down-steps staying above diagonal.

Super-Catalan Numbers

Also known as little Schröder numbers. Count similar objects with an additional type of step.

Narayana Numbers

Refinement of Catalan numbers that also track number of peaks.

16. Related Concepts

- **Dyck words** — balanced parentheses strings
- **nCr** — binomial coefficient used in Catalan formula
- **Schröder numbers** — related combinatorial sequence
- **Bell numbers** — count partitions, different from Catalan

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Unique Binary Search Trees	LeetCode 96	C_n different BSTs with n nodes	Medium (but easy with Catalan)
Generate Parentheses	LeetCode 22	Generate all C_n valid strings	Medium

Medium

Problem	Platform	Main Idea	Difficulty
Different Ways to Add Parentheses	LeetCode 241	Catalan-like DP	Medium
Number of Ways to Stay in Place	LeetCode 1269	Paths with steps = Catalan-like	Hard

Hard

Problem	Platform	Main Idea	Difficulty
Catalan Square	Codeforces	$C_n \times C_n$ type convolutions	Hard
Triangulations	CSES	Polygon triangulation count	Hard

18. Interview Explanation

"Catalan numbers count structures that can be recursively split in two. The n -th Catalan number is $C_n = \frac{C(2n, n)}{n+1}$. They count valid parentheses expressions, binary trees, and monotonic paths. I compute them using $nCr(2n, n) * inv(n+1) \% MOD$ with precomputed factorials. The recurrence $C_{n+1} = \sum C_i \times C_{n-i}$ shows the recursive splitting structure."

19. Revision Notes

- **Formula:** $C_n = \frac{(2n)!}{(n! \times (n+1)!)} = \frac{C(2n, n)}{n+1}$
- **Recurrence:** $C_0 = 1, C_{n+1} = \sum C_i \times C_{n-i}$
- **Computations:** Use formula for modulo, DP for small exact values
- **Common interpretations:** Parentheses, binary trees, Dyck paths, triangulations
- **Growth:** $\sim 4^n / (n^{3/2} \times \sqrt{\pi})$

20. Final Cheat Sheet

Aspect	Details
Formula	$C_n = C(2n, n) / (n+1)$
Recurrence	$C_0 = 1, C_n = \sum C_i \times C_{n-1-i}$
Identical to	Valid parentheses, BST count, Dyck paths
Computation	$nCr(2n, n) * inv(n+1) \% MOD$
Complexity	$O(n)$ with precomputed factorials
Growth	Exponential ($\sim 4^n$)

19. Matrix Exponentiation

1. Overview

Matrix exponentiation applies binary exponentiation to matrices. Used to compute the n -th term of linear recurrences (like Fibonacci) in $O(k^3 \log n)$, where k is the recurrence order.

2. Intuition

Core idea: Represent a linear recurrence as a matrix transformation. Computing the n -th term is equivalent to applying the transformation n times $\rightarrow$ raise the matrix to the n -th power using binary exponentiation.

Analogy: If you have a transformation rule T that advances one step, applying it n times is T^n . Instead of applying it n times ($O(n)$), compute T^n via exponentiation ($O(\log n)$).

Why it works: Linear recurrences like $F_n = F_{n-1} + F_{n-2}$ can be written as:

$$\begin{bmatrix} F_n \\ F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \times \begin{bmatrix} F_{n-1} \\ F_{n-2} \end{bmatrix}$$

If M is the transformation matrix, then $M^n \times [F_1, F_0]^T = [F_{n+1}, F_n]^T$.

3. When to Use It

- Computing n -th term of a linear recurrence (Fibonacci, Tribonacci, etc.)
- Any DP that can be modeled as a linear transformation
- Counting walks in a graph (number of length- k paths between nodes)
- Problems where n is huge (10^{18}) and recurrence order is small ($k \leq 100$)
- Computing transformations that are linear and associative

4. When Not to Use It

- When recurrence order is very large ($k > 500$) — $O(k^3 \log n)$ becomes expensive
- When the recurrence is not linear (has non-linear terms like F_n^2)
- When $O(n)$ DP is fast enough ($n \leq 10^7$)
- When you can use the closed-form formula (e.g., Binet's for Fibonacci — but matrix exp is often easier to implement with modulo)

5. Core Concepts

Recurrence Matrix

For $F_n = c_1 F_{n-1} + c_2 F_{n-2} + \dots + c_k F_{n-k}$, the companion matrix is:

$$\begin{vmatrix} c_1 & c_2 & \dots & c_{k-1} & c_k \\ 1 & 0 & \dots & 0 & 0 \\ 0 & 1 & \dots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & 1 & 0 \end{vmatrix}$$

Binary Exponentiation for Matrices

Same as integer exponentiation: square the matrix at each step, multiply by base when bit is 1.

Identity Matrix

I such that $M \times I = M$. The "1" of matrix multiplication.

Associativity

Matrix multiplication is associative: $(A \times B) \times C = A \times (B \times C)$. Essential for binary exponentiation.

6. Step-by-Step Algorithm

1. Build the $k \times k$ transformation matrix M .
2. Build the base vector V (initial terms).
3. Compute M^n using binary exponentiation.
4. Multiply $M^n \times V$ to get result.
5. Extract the desired term.

7. Dry Run — Fibonacci

Find F_{10} using matrix exponentiation.

Recurrence: $F_n = F_{n-1} + F_{n-2}$, $F_0 = 0$, $F_1 = 1$.

Transformation matrix $M = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$.

We want $M^{10} \times [F_1, F_0]^T = [F_{11}, F_{10}]^T$.

Compute M^{10} via binary exponentiation ($10 = 1010_2$):

Step	Base (M^{power})	Result	Bits of 10
0	$M^1 = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$	I	0
1	$M^2 = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$	M^2	1
2	$M^4 = \begin{bmatrix} 3 & 2 \\ 2 & 1 \end{bmatrix}$	M^2	0
3	$M^8 = \begin{bmatrix} 5 & 3 \\ 3 & 2 \end{bmatrix}$	$M^2 \times M^8 = M^{10}$	1

$M^{10} \times [1, 0]^T = \begin{bmatrix} 55 & 34 \\ 34 & 21 \end{bmatrix} \times [1, 0]^T = [55, 34]^T$.

So $F_{10} = 34$. ✓ (Fibonacci: 0,1,1,2,3,5,8,13,21,34,...)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const int MOD = 1e9 + 7;

using Matrix = vector<vector<ll>>>;

Matrix matMul(const Matrix& A, const Matrix& B) {
    int n = A.size(), m = B[0].size(), k = A[0].size();
    Matrix C(n, vector<ll>(m, 0));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            for (int t = 0; t < k; t++) {
                C[i][j] = (C[i][j] + A[i][t] * B[t][j]) % MOD;
            }
        }
    }
    return C;
}

Matrix matPow(Matrix M, ll exp) {
    int n = M.size();
    Matrix res(n, vector<ll>(n, 0));
    for (int i = 0; i < n; i++) res[i][i] = 1; // identity
```

```

    while (exp > 0) {
        if (exp & 1) res = matMul(res, M);
        M = matMul(M, M);
        exp >>= 1;
    }
    return res;
}

// Fibonacci: F_n with matrix exponentiation
ll fibonacciMat(ll n) {
    if (n == 0) return 0;
    if (n == 1) return 1;

    Matrix M = {{1, 1}, {1, 0}};
    Matrix Mn = matPow(M, n - 1);
    // Mn * [F1, F0]^T = [Fn, Fn-1]^T
    // F1 = 1, F0 = 0
    return (Mn[0][0] * 1 + Mn[0][1] * 0) % MOD;
}

// General linear recurrence: F(n) = c1*F(n-1) + c2*F(n-2) + ... + ck*F(n-k)
ll linearRecurrence(vector<ll> coeffs, vector<ll> initial, ll n) {
    int k = coeffs.size();
    if (n < k) return initial[n];

    // Build companion matrix
    Matrix M(k, vector<ll>(k, 0));
    for (int i = 0; i < k; i++) M[0][i] = coeffs[i];
    for (int i = 1; i < k; i++) M[i][i - 1] = 1;

    Matrix Mn = matPow(M, n - k + 1);

    // Result = Mn x [initial[k-1], ..., initial[0]]^T
    ll res = 0;
    for (int i = 0; i < k; i++) {
        res = (res + Mn[0][i] * initial[k - 1 - i]) % MOD;
    }
    return res;
}

int main() {
    cout << fibonacciMat(10) << "\n"; // 34 (F10)
    cout << fibonacciMat(100) << "\n"; // large, works fast

    // Tribonacci: F(n) = F(n-1) + F(n-2) + F(n-3), F0=0, F1=1, F2=1
    vector<ll> coeffs = {1, 1, 1};
    vector<ll> init = {0, 1, 1};
    cout << linearRecurrence(coeffs, init, 10) << "\n";
    return 0;
}

```

9. Python Implementation

```
MOD = 10**9 + 7

def mat_mul(A, B):
    n, m, k = len(A), len(B[0]), len(A[0])
    C = [[0] * m for _ in range(n)]
    for i in range(n):
        for j in range(m):
            for t in range(k):
                C[i][j] = (C[i][j] + A[i][t] * B[t][j]) % MOD
    return C

def mat_pow(M, exp):
    n = len(M)
    res = [[1 if i == j else 0 for j in range(n)] for i in range(n)]
    while exp:
        if exp & 1:
            res = mat_mul(res, M)
        M = mat_mul(M, M)
        exp >>= 1
    return res

def fibonacci_mat(n):
    if n == 0: return 0
    M = [[1, 1], [1, 0]]
    Mn = mat_pow(M, n - 1)
    return (Mn[0][0] * 1 + Mn[0][1] * 0) % MOD

print(fibonacci_mat(10)) # 34
print(fibonacci_mat(100))
```

10. Code Explanation

- **matMul**: Standard $O(k^3)$ matrix multiplication. Three nested loops with modulo.
- **matPow**: Same binary exponentiation logic as integer version, but with matrix multiplication.
- **Identity matrix**: Diagonal of 1s — acts as multiplicative identity.
- **Fibonacci**: $M = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$. $M^{(n-1)}$ gives F_n directly.
- **General recurrence**: Build companion matrix from coefficients. Multiply by initial value vector.

11. Complexity Analysis

Operation	Time	Space
Linear recurrence (order k)	$O(k^3 \log n)$	$O(k^2)$
Fibonacci (order 2)	$O(8 \log n) \approx O(\log n)$	$O(4)$
Matrix multiplication	$O(k^3)$	$O(k^2)$

12. Common Patterns

Pattern	How to Identify	Matrix Construction
Fibonacci	$F_n = F_{n-1} + F_{n-2}$	$[[1,1],[1,0]]$
General linear recurrence	$F_n = c_1 F_{n-1} + \dots + c_k F_{n-k}$	Companion matrix (first row = coeffs, sub-diagonal = 1)
Graph walks	"number of length-k paths between nodes"	Adjacency matrix
DP with linear transition	$DP[i] = \sum c_j \times DP[i-j]$	Companion matrix

13. Common Mistakes

- **Wrong base/initial terms:** Ensure the base vector is correctly ordered (latest term first or last depending on convention).
- **Exponent off-by-one:** For Fibonacci, $M^{(n-1)}$ gives F_n . For a recurrence with k terms, $M^{(n-k+1)}$.
- **Matrix multiplication is NOT commutative:** Order matters: $A \times B \neq B \times A$.
- **Incorrect matrix dimensions:** Companion matrix is $k \times k$. Result vector is $k \times 1$.
- **Missing modulo:** Without modulo, values overflow quickly.

14. Edge Cases

n	Expected	Notes
0	$F_0 = 0$	Handle base cases first
1	$F_1 = 1$	Don't apply matrix for $n < k$
Large n (10^{18})	Works in $O(\log n)$	Matrix exponentiation handles

15. Variations

Linear Recurrence with Constant Term

If $F_n = c_1 F_{n-1} + \dots + c_k F_{n-k} + d$, augment the matrix to include the constant.

Fast Fibonacci Using 2×2 Scaled

Optimized 2×2 multiplication with only 3 multiplications (Strassen-like) for Fibonacci.

Sparse Matrix Multiplication

If the transformation matrix is sparse (mostly zeros), use sparse multiplication for $O(k^2 \log n)$.

16. Related Techniques

- **Binary exponentiation** — integer version of the same idea
- **Fast doubling for Fibonacci** — $O(\log n)$ without matrices, works only for Fibonacci

- **Generating functions** — alternative theoretical approach to solving recurrences
- **Berlekamp-Massey** — find the linear recurrence from terms

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Fibonacci Number	LeetCode 509	Matrix exponentiation for Fibonacci	Easy
Climbing Stairs	LeetCode 70	Fibonacci recurrence	Easy

Medium

Problem	Platform	Main Idea	Difficulty
Tiling Dominoes	LeetCode 1240	$2 \times n$ tiling uses matrix exponentiation	Medium
Number of Ways	Codeforces	DP $\rightarrow$ matrix exponentiation	Medium

Hard

Problem	Platform	Main Idea	Difficulty
Count Ways to Build Cells	Codeforces	Complex linear recurrence	Hard
Matrix Exponentiation	CSES	Fibonacci with queries	Hard

18. Interview Explanation

"Matrix exponentiation computes the n -th term of a linear recurrence in $O(k^3 \log n)$ where k is the order. I represent the recurrence as a companion matrix M , then compute M^n using binary exponentiation. For Fibonacci, $M = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$, and $M^{(n-1)}$ gives F_n . Each multiplication costs $O(k^3)$, and we do $O(\log n)$ multiplications. This is essential when n is huge (10^{18}) but the recurrence order is small."

19. Revision Notes

- **Recurrence $\rightarrow$ Matrix:** $F_n = \sum c_i F_{n-i} \rightarrow$ companion matrix with first row $c_1 \dots c_k$, sub-diagonal 1
- **Result:** $M^n \times \text{initial_vector} \rightarrow$ get desired term
- **Complexity:** $O(k^3 \log n)$
- **Key code:** matPow similar to binary exponentiation but with matrix multiplication
- **Edge:** Handle base cases ($n < k$) separately

20. Final Cheat Sheet

Aspect	Details
When to use	Linear recurrence with huge n (up to 10^{18})

Aspect	Details
Algorithm	Build companion matrix → matPow → multiply with initial vector
Complexity	$O(k^3 \log n)$ where k = order of recurrence
Key matrix	Companion: first row = coefficients, rest is sub-diagonal of 1s
Key code	<code>matPow(M, n) @ initial_vec</code>
Edge	$n < k \rightarrow$ just return initial[n]

20. Mobius Function (μ)

1. Overview

The Möbius function $\mu(n)$ is defined for positive integers:

- $\mu(1) = 1$
- $\mu(n) = 0$ if n has a squared prime factor
- $\mu(n) = (-1)^k$ if n is product of k distinct primes

2. Intuition

Why it matters: The Möbius function serves as the "inverse" of the constant function 1 in the context of divisor sums. Möbius inversion allows us to convert sums over divisors into simpler forms.

Analogy: In calculus, differentiation and integration are inverse operations. In number theory, the Möbius function plays a similar role — it inverts summations over divisors.

Key property: $\sum_{d|n} \mu(d) = 1$ if $n = 1$, else 0.

3. When to Use It

- Möbius inversion: converting sum-over-divisors formulas
- Counting coprime pairs in ranges
- Inclusion-Exclusion over divisor lattice
- Computing Euler totient via Möbius ($\varphi(n) = \sum_{d|n} \mu(d) \times n/d$)
- Counting square-free numbers
- Analyzing Dirichlet convolutions

4. When Not to Use It

- When you need $\mu(n)$ for a single value (just compute directly from prime factorization)
- When the range is small (inclusion-exclusion via bitmask is simpler for $n \leq 15-20$)
- When the divisor lattice is not involved

5. Core Concepts

Definition

See above. $\mu(n)$ detects whether n is square-free and how many prime factors it has.

Möbius Inversion

If $f(n) = \sum_{d|n} g(d)$, then $g(n) = \sum_{d|n} \mu(d) \times f(n/d)$.

Why it matters: Allows conversion between two forms of sum-over-divisors.

Precomputation via Sieve

$\mu(n)$ can be precomputed for all n up to N using a linear sieve ($O(N)$).

Critical Identity

$\sum_{d|g} \mu(d) = [g = 1]$ (Iverson bracket: 1 if $g = 1$, 0 otherwise).

This is the foundation for most Möbius-based counting.

6. Step-by-Step Algorithm

Computing $\mu(n)$ for a single n :

- 1. Factorize n .
- 2. If any exponent > 1 : $\mu(n) = 0$.
- 3. If k distinct primes: $\mu(n) = (-1)^k$.

Precomputing μ for all $n \leq N$:

- 1. Initialize $\mu[1] = 1$.
- 2. Use modified linear sieve:
 - For each prime p : $\mu[p] = -1$.
 - For $i \times p$: if $p \nmid i$, $\mu[i \times p] = -\mu[i]$; else $\mu[i \times p] = 0$.

7. Dry Run

Precompute $\mu[1..12]$:

n	Factorization	Square-free	Distinct primes	$\mu(n)$
1	—	Yes	0	1
2	2^1	Yes	1	-1
3	3^1	Yes	1	-1
4	2^2	No	—	0
5	5^1	Yes	1	-1
6	2×3	Yes	2	1
7	7^1	Yes	1	-1
8	2^3	No	—	0

n	Factorization	Square-free	Distinct primes	$\mu(n)$
9	3^2	No	—	0
10	2×5	Yes	2	1
11	11^1	Yes	1	-1
12	$2^2 \times 3$	No	—	0

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Compute mu(n) for a single n
int mobiusSingle(int n) {
    if (n == 1) return 1;
    int cnt = 0;
    for (int p = 2; p * p <= n; p++) {
        if (n % p == 0) {
            int exp = 0;
            while (n % p == 0) { n /= p; exp++; }
            if (exp > 1) return 0;
            cnt++;
        }
    }
    if (n > 1) cnt++;
    return (cnt % 2 == 0) ? 1 : -1;
}

// Precompute mu for all numbers up to N using linear sieve
const int MAXN = 1000000;
int mu[MAXN + 1];
vector<int> primes;
bool isComposite[MAXN + 1];

void precomputeMobius(int n = MAXN) {
    mu[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (!isComposite[i]) {
            primes.push_back(i);
            mu[i] = -1; // prime: -1
        }
        for (int p : primes) {
            if (i * p > n) break;
            isComposite[i * p] = true;
            if (i % p == 0) {
                mu[i * p] = 0; // squared factor
                break;
            }
            mu[i * p] = -mu[i]; // new prime factor → flip sign
        }
    }
}
```

```

    }
}

// Count coprime pairs (a,b) with 1 ≤ a,b ≤ n
// Using:  $\sum_{d=1}^n \mu(d) \times (n/d)^2$ 
long long countCoprimePairs(int n) {
    long long ans = 0;
    for (int d = 1; d <= n; d++) {
        ans += (long long)mu[d] * (n / d) * (n / d);
    }
    return ans;
}

int main() {
    precomputeMobius(20);
    for (int i = 1; i <= 12; i++) {
        cout << "mu[" << i << "] = " << mu[i] << "\n";
    }
    // mu[1]=1, mu[2]=-1, mu[3]=-1, mu[4]=0, mu[5]=-1, mu[6]=1,
    // mu[7]=-1, mu[8]=0, mu[9]=0, mu[10]=1, mu[11]=-1, mu[12]=0

    cout << "Coprime pairs up to 5: " << countCoprimePairs(5) << "\n";
    return 0;
}

```

9. Python Implementation

```

def mobius_single(n: int) -> int:
    if n == 1:
        return 1
    cnt = 0
    p = 2
    while p * p <= n:
        if n % p == 0:
            exp = 0
            while n % p == 0:
                n //= p
                exp += 1
            if exp > 1:
                return 0
            cnt += 1
        p += 1 if p == 2 else 2
    if n > 1:
        cnt += 1
    return 1 if cnt % 2 == 0 else -1

def precompute_mobius(n: int):
    mu = [0] * (n + 1)
    mu[1] = 1
    primes = []
    is_comp = [False] * (n + 1)

```

```
for i in range(2, n + 1):
    if not is_comp[i]:
        primes.append(i)
        mu[i] = -1
    for p in primes:
        if i * p > n:
            break
        is_comp[i * p] = True
        if i % p == 0:
            mu[i * p] = 0
            break
        mu[i * p] = -mu[i]
return mu

mu = precompute_mobius(12)
for i in range(1, 13):
    print(f"mu[{i}] = {mu[i]}")
```

10. Code Explanation

- **Single $\mu(n)$:** Factorize n . If any exponent > 1 , return 0. Otherwise, $\mu = (-1)^{(\text{number of distinct primes})}$.
- **Sieve precomputation:** Linear sieve (each composite marked exactly once). When marking $i \times p$:
 - If $i \% p == 0$: p appears again in $i \times p$, so p^2 divides $i \times p \rightarrow \mu = 0$.
 - Otherwise: $\mu(i \times p) = -\mu(i)$ (adding one more distinct prime flips sign).
- **Counting coprime pairs:** Using Möbius inversion on the divisor sum. Numbers with $\text{gcd}(a,b) = d$ are counted by $\mu(d)$.

11. Complexity Analysis

Operation	Time Complexity	Space
Single $\mu(n)$	$O(\sqrt{n})$	$O(1)$
Precompute μ (sieve)	$O(N)$	$O(N)$
Möbius inversion	$O(n)$ or $O(\sqrt{n})$	$O(1)$

12. Common Patterns

Pattern	How to Identify	Approach
Count coprime pairs (1..n)	"gcd = 1", "coprime"	$\sum \mu(d) \times (n/d)^2$
Sum over divisors	$\sum_{d n}$	$\sum_{d n} f(d)$
Square-free numbers	"product of distinct primes"	Count via $\mu^2(n) =$
Inclusion-exclusion over primes	"numbers not divisible by any prime in set"	Use $\mu(d)$ as coefficients

13. Common Mistakes

- **$\mu(1) = 1$** : Many forget this base case.
- **Confusing μ with τ or σ** : μ is NOT the divisor count or sum.
- **Incorrect sieve logic**: When $i \% p == 0$, $\mu(i \times p) = 0$ AND we break (don't continue marking with larger primes).
- **Integer overflow**: In counting formulas, $(n/d)^2$ can overflow int. Use long long.

14. Edge Cases

n	$\mu(n)$	Reason
1	1	By definition
p	-1	Single prime
$p \times q$	1	Two distinct primes
p^2	0	Squared factor
$p \times q \times r$	-1	Three distinct primes

15. Variations

Mertens Function

$M(n) = \sum_{i=1}^n \mu(i)$. Used in prime number theory for analyzing prime density.

Möbius Inversion on General Posets

Generalization to partially ordered sets, not just divisor lattice.

Square-Free Counting

The number of square-free numbers $\leq n$ is $\sum \mu^2(i)$. Approximately $6n/\pi^2$.

16. Related Functions

- **Divisor function $\tau(n)$** — number of divisors
- **Sum-of-divisors $\sigma(n)$** — sum of divisors
- **Euler totient $\varphi(n)$** — count of coprime numbers $\leq n$
- **Liouville function $\lambda(n)$** = $(-1)^{(\text{total prime factors with multiplicity})}$

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Möbius Function	GFG	Direct computation	Easy

Medium

Problem	Platform	Main Idea	Difficulty
Coprime Pairs	Codeforces	Count pairs with $\gcd = 1$	Medium
Möbius Inversion	CSES	Sum over divisors	Medium

Hard

Problem	Platform	Main Idea	Difficulty
Counting GCD	Codeforces	Advanced Möbius with range	Hard
Möbius Sum	Project Euler	Large-scale sums	Hard

18. Interview Explanation

"The Möbius function $\mu(n)$ is 1 if $n=1$, $(-1)^k$ if n has k distinct prime factors, and 0 if n has a squared prime factor. Its key property is $\sum_{d|n} \mu(d) = 1$ if $n=1$, else 0. This makes it the inverse of the constant function 1 under Dirichlet convolution, enabling Möbius inversion. I precompute it using a linear sieve in $O(N)$. A classic application is counting coprime pairs: $\sum \mu(d) \times (N/d)^2$."

19. Revision Notes

- $\mu(1) = 1$
- $\mu(n) = 0$ if any prime exponent > 1
- $\mu(n) = (-1)^k$ if n has k distinct primes
- **Key identity:** $\sum_{d|n} \mu(d) = [n = 1]$
- **Precomputation:** Linear sieve $O(N)$
- **Application:** Counting coprime pairs, Möbius inversion

20. Final Cheat Sheet

Aspect	Details
Definition	$\mu(1)=1$, $\mu(n)=0$ if squared factor, otherwise $(-1)^k$
Key identity	$\sum_{d n} \mu(d)$
Precompute	Linear sieve $O(N)$
Application	Counting coprime pairs, inclusion-exclusion
Complexity	$O(N)$ precomputation, $O(\sqrt{n})$ per value
Formula	Count coprime = $\sum \mu(d) \times (N/d)^2$

21. Euler Totient Function (φ)

1. Overview

Euler's totient function $\varphi(n)$ counts positive integers $\leq n$ that are coprime to n . $\varphi(1) = 1$.

For $n = p_1^{a_1} \times p_2^{a_2} \times \dots \times p_k^{a_k}$: $\phi(n) = n \times (1 - 1/p_1) \times (1 - 1/p_2) \times \dots \times (1 - 1/p_k)$

2. Intuition

Analogy: Among numbers 1 to n , what fraction are NOT divisible by any prime factor of n ? If we remove multiples of each prime factor, using inclusion-exclusion, we get $\phi(n)/n = \prod (1 - 1/p)$.

Why it matters: $\phi(n)$ appears in Euler's theorem (generalization of Fermat): $a^{\phi(n)} \equiv 1 \pmod{n}$ when $\gcd(a, n) = 1$. This is fundamental for RSA and modular arithmetic.

3. When to Use It

- Counting numbers coprime to n in range $[1, n]$
- In Euler's theorem for modular arithmetic when modulus is not prime
- In RSA key generation (computing $\phi(N)$ where $N = p \times q$)
- In problems involving primitive roots
- Computing power towers modulo composite
- As part of other number-theoretic computations (e.g., divisor sums)

4. When Not to Use It

- When you only need count of coprime numbers up to a fixed N for all values (use sieve)
- When modulus is prime (Fermat's theorem is simpler)
- When $\gcd(a, n) \neq 1$ (Euler's theorem doesn't apply directly)

5. Core Concepts

Formula

For $n = \prod p_i^{a_i}$: $\phi(n) = n \times \prod (1 - 1/p_i)$.

Euler's Theorem

For $\gcd(a, n) = 1$: $a^{\phi(n)} \equiv 1 \pmod{n}$.

Multiplicativity

If $\gcd(m, n) = 1$: $\phi(m \times n) = \phi(m) \times \phi(n)$.

$$\phi(p) = p - 1$$

For a prime p , all numbers $1..p-1$ are coprime to p .

$$\phi(p^k) = p^k - p^{k-1} = p^k \times (1 - 1/p)$$

6. Step-by-Step Algorithm

Compute $\phi(n)$ for a single n :

1. Factorize n .
2. result = n .

3. For each distinct prime factor p : $\text{result} -= \text{result} / p$.
4. Return result.

Precompute ϕ for all numbers up to N :

1. Initialize $\text{phi}[i] = i$.
2. For each prime p : for multiples of p : $\text{phi}[\text{multiple}] -= \text{phi}[\text{multiple}] / p$.

7. Dry Run

Compute $\phi(12)$:

$$n = 12 = 2^2 \times 3. \phi(12) = 12 \times (1 - 1/2) \times (1 - 1/3) = 12 \times 1/2 \times 2/3 = 4.$$

Verify: Numbers ≤ 12 coprime to 12: $\{1, 5, 7, 11\} = 4$ numbers $\checkmark$.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

//  $\phi(n)$  for single n (using prime factorization)
ll phiSingle(ll n) {
    ll result = n;
    ll temp = n;
    for (ll p = 2; p * p <= temp; p++) {
        if (temp % p == 0) {
            while (temp % p == 0) temp /= p;
            result -= result / p;
        }
    }
    if (temp > 1) result -= result / temp;
    return result;
}

// Precompute  $\phi$  for all numbers up to N
const int MAXN = 1000000;
int phi[MAXN + 1];

void precomputePhi(int n = MAXN) {
    for (int i = 0; i <= n; i++) phi[i] = i;
    for (int i = 2; i <= n; i++) {
        if (phi[i] == i) { // i is prime
            for (int j = i; j <= n; j += i) {
                phi[j] -= phi[j] / i;
            }
        }
    }
}
```

```

// Euler's theorem:  $a^{\phi(n)} \equiv 1 \pmod n$  (if  $\gcd(a,n)=1$ )
ll modPow(ll a, ll b, ll mod) {
    ll res = 1; a %= mod;
    while (b) { if (b & 1) res = res * a % mod; a = a * a % mod; b >>= 1; }
    return res;
}

// Using Euler's theorem to reduce exponent:  $a^b \pmod n$ 
// where b can be very large (string)
ll eulerReduce(ll a, string b, ll n) {
    ll phiVal = phiSingle(n);
    ll exp = 0;
    for (char c : b) {
        exp = (exp * 10 + (c - '0')) % phiVal;
    }
    // If b < phi(n), we need the actual exponent, not reduced
    // This simplified version works when  $\gcd(a,n) = 1$ 
    if (exp == 0) exp = phiVal; // Adjust for the +0 case
    return modPow(a, exp, n);
}

int main() {
    cout << phiSingle(12) << "\n"; // 4
    cout << phiSingle(100) << "\n"; // 40

    precomputePhi(20);
    for (int i = 1; i <= 12; i++) {
        cout << "phi[" << i << "] = " << phi[i] << "\n";
    }
    // 1 1 2 2 4 2 6 4 6 4 10 4

    return 0;
}

```

9. Python Implementation

```

def phi_single(n: int) -> int:
    result = n
    p = 2
    temp = n
    while p * p <= temp:
        if temp % p == 0:
            while temp % p == 0:
                temp //= p
            result -= result // p
        p += 1 if p == 2 else 2
    if temp > 1:
        result -= result // temp
    return result

def precompute_phi(n: int):

```

```

phi = list(range(n + 1))
for i in range(2, n + 1):
    if phi[i] == i: # i is prime
        for j in range(i, n + 1, i):
            phi[j] -= phi[j] // i
return phi

print(phi_single(12)) # 4
print(phi_single(100)) # 40

phi = precompute_phi(12)
for i in range(1, 13):
    print(f"phi[{i}] = {phi[i]}")

```

10. Code Explanation

- **Single ϕ :** Factorize n , apply formula `result -= result/p` for each distinct prime.
- **Precomputation via sieve:** Start with `phi[i] = i`. For each prime i , subtract `phi[j] / i` from all multiples j . This effectively multiplies by $(1 - 1/i)$.
- **Euler's theorem for exponent reduction:** Since $a^{\phi(n)} \equiv 1 \pmod{n}$, for large exponent b , we reduce it modulo $\phi(n)$.

11. Complexity Analysis

Operation	Time Complexity	Space
Single $\phi(n)$	$O(\sqrt{n})$	$O(1)$
Precompute ϕ up to N	$O(N \log \log N)$	$O(N)$
Euler reduction	$O(\log b)$	$O(1)$

12. Common Patterns

Pattern	How to Identify	Approach
Count coprime to n	"numbers coprime to n "	Direct $\phi(n)$ formula
Sum of coprime numbers $\leq n$	"sum of k where $\gcd(k, n) = 1$ "	Sum = $n \times \phi(n) / 2$ (for $n > 1$)
Euler exponent reduction	" $a^b \pmod{n}$ where b is huge"	Reduce exponent mod $\phi(n)$
RSA totient	" p and q primes, find $\phi(p \times q)$ "	$\phi(pq) = (p-1) \times (q-1)$
Period of repeating fraction	"repeating decimal period length"	min d such that $10^d \equiv 1 \pmod{n}$

13. Common Mistakes

- **Not dividing by each distinct prime:** The formula needs distinct primes only, not prime powers.
- **Forgetting $n = 1$:** $\phi(1) = 1$ by convention.
- **Using Euler's theorem when $\gcd \neq 1$:** $a^{\phi(n)} \equiv 0 \pmod{n}$ may not hold. Use Carmichael function for general case.

- **Exponent reduction edge case:** When the reduced exponent is 0, use $\varphi(n)$ instead (because $a^{(k\varphi(n))} \equiv 1$).
- **Overflow in precomputation:** $\text{phi}[j] - \text{phi}[j]/i$ is safe with ints.

14. Edge Cases

n	$\varphi(n)$	Reason
1	1	By definition
p	p-1	Prime — all smaller are coprime
p^k	$p^k - p^{(k-1)}$	Remove multiples of p
2p	p-1 (if p > 2)	$\varphi(2p) = \varphi(2) \times \varphi(p) = 1 \times (p-1)$
large prime product	$(p-1)(q-1)$	RSA

15. Variations

Carmichael Function $\lambda(n)$

Smallest exponent such that $a^{\lambda(n)} \equiv 1 \pmod n$ for all coprime a. $\lambda(n) \leq \varphi(n)$ and often divides $\varphi(n)$.

Jordan's Totient

Generalization of φ to count k-tuples of coprime numbers.

Divisor Sum of φ

$\sum_{d|n} \varphi(d) = n$.

16. Related Functions

- **Möbius function $\mu(n)$** — related via $\varphi(n) = \sum_{d|n} \mu(d) \times (n/d)$
- **Divisor sum $\sigma_0(n)$** — φ relates to divisor counts
- **Mertens function** — sum of Möbius for prime counting

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Euler Totient	GFG	Direct $\varphi(n)$	Easy
φ of n	LeetCode (variant)	Formula application	Easy

Medium

Problem	Platform	Main Idea	Difficulty
RSA Decryption	LeetCode (variant)	$\varphi(pq)$ for RSA	Medium

Problem	Platform	Main Idea	Difficulty
Coprime Count	CSES	Range totient	Medium

Hard

Problem	Platform	Main Idea	Difficulty
Totient Sum	SPOJ	Large-scale totient sums	Hard
Euler Totient with Queries	Codeforces	Precomputation + queries	Hard

18. Interview Explanation

"Euler's totient $\phi(n)$ counts numbers $\leq n$ coprime to n . For $n = \prod p_i^{a_i}$, $\phi(n) = n \times \prod (1 - 1/p_i)$. I compute it by factorizing n and subtracting result/p for each distinct prime. Euler's theorem $a^{\phi(n)} \equiv 1 \pmod n$ for $\gcd(a,n)=1$ generalizes Fermat's theorem, allowing exponent reduction modulo $\phi(n)$. I can precompute ϕ for all numbers up to N using a sieve in $O(N \log \log N)$."

19. Revision Notes

- **Formula:** $\phi(n) = n \times \prod (1 - 1/p_i)$ for distinct primes p_i
- **$\phi(p) = p-1$, $\phi(p^k) = p^k - p^{k-1}$**
- **Multiplicative:** $\phi(mn) = \phi(m) \times \phi(n)$ when $\gcd(m,n)=1$
- **Euler's theorem:** $a^{\phi(n)} \equiv 1 \pmod n$ if $\gcd(a,n)=1$
- **Identity:** $\sum_{d|n} \phi(d) = n$
- **Sum of coprime numbers $\leq n$:** $n \times \phi(n) / 2$ for $n > 1$

20. Final Cheat Sheet

Aspect	Details
Formula	$\phi(n) = n \times \prod (1 - 1/p)$
Computation	Factorize n , for each distinct prime p : $\text{result} -= \text{result} / p$
Precompute	Sieve $O(N \log \log N)$: $\phi[i] -= \phi[i] / i$ for all multiples
Euler's theorem	$a^{\phi(n)} \equiv 1 \pmod n$ when $\gcd(a,n)=1$
Applications	RSA, exponent reduction, coprime counting
Identity	$\sum \phi(d) = n$ over d

Maths-for-DSA Quick Reference

Complexity at a Glance

Algorithm/Section	Time Complexity	Key Feature
GCD (Euclidean)	$O(\log \min(a,b))$	while(b) swap(a%=b, b)
LCM	$O(\log \min(a,b))$	$a / \gcd * b$ (divide first!)
Prime Checking	$O(\sqrt{n})$	$6k \pm 1$ optimization
Sieve of Eratosthenes	$O(n \log \log n)$	Mark from p^2 , step p
Modular Arithmetic	$O(1)$ per op	Negatives: $(x \% M + M) \% M$
Modular Exponentiation	$O(\log b)$	Binary exponentiation
Factorial Modulo	$O(n)$ pre, $O(1)$ query	$nCr = \text{fact}[n] \times \text{invFact}[r] \times \text{invFact}[n-r]$
Divisors	$O(\sqrt{n})$	Collect pairs (i, n/i)
Prime Factorization	$O(\sqrt{n})$	Divide by 2, then odds only
Modular Inverse	$O(\log M)$	Fermat (prime) or Extended Euclidean
Extended Euclidean	$O(\log \min(a,b))$	Returns (g,x,y) for $ax+by=g$
CRT	$O(k \log M)$	Pairwise combining
Inclusion-Exclusion	$O(2^n)$	Bitmask iteration
Catalan Numbers	$O(n)$	$C(2n,n) / (n+1)$
Matrix Exponentiation	$O(k^3 \log n)$	Companion matrix
Möbius Function	$O(N)$ sieve	Linear sieve for $\mu[1..N]$
Euler Totient	$O(\sqrt{n})$ single, $O(N \log \log N)$ pre	$n \times \prod (1 - 1/p)$

Key Formulas Cheat Sheet

- **GCD:** $\gcd(a, b) \times \text{lcm}(a, b) = a \times b$
- **nCr:** $C(n, r) = n! / (r! \times (n-r)!)$
- **nPr:** $P(n, r) = n! / (n-r)!$
- **Catalan:** $C_n = C(2n, n) / (n+1)$
- **Euler Totient:** $\varphi(n) = n \times \prod (1 - 1/p_i)$
- **Möbius:** $\sum_{d|n} \mu(d) = [n=1]$
- **Fermat:** $a^{(p-1)} \equiv 1 \pmod{p}$ for prime p
- **Euler:** $a^{\varphi(n)} \equiv 1 \pmod{n}$ for $\gcd(a,n)=1$
- **Modular Inverse:** $a^{-1} \equiv a^{(M-2)} \pmod{M}$ for prime M
- **Pascal:** $C(n, k) = C(n-1, k-1) + C(n-1, k)$
- **Stars and Bars:** $C(n+r-1, r-1)$ ways to distribute n identical items into r boxes
- **Binomial Theorem:** $(a+b)^n = \sum C(n,k) \times a^k \times b^{(n-k)}$

Common Modula in CP

Modulus	Properties	Used In
1,000,000,007 (1e9+7)	Prime, fits in 32-bit	Most platforms
998,244,353	Prime, NTT-friendly	Codeforces
1,000,000,009 (1e9+9)	Prime	Codeforces, hashing
10^{18}	Not prime	Safe multiplication needed

End of Guide — Master these 21 topics for placement and CP success.